



UQ24CA342A Software Testing and Automation

Mrs. Sowmya BP

Department of Computer Applications

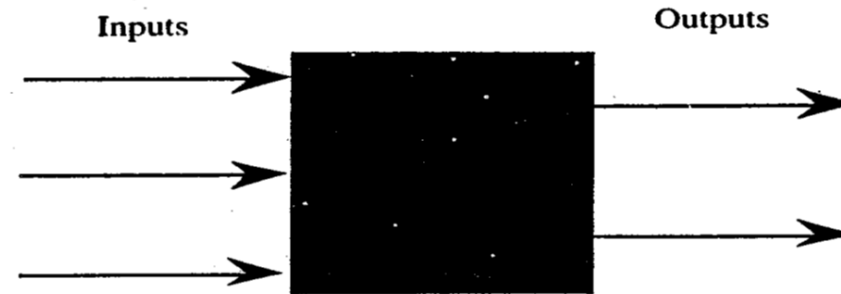
UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

Department of Computer Applications

Black-box testing is a software testing technique in which the internal workings of the system being tested are not known to the tester



Advantages:

- It enables testers to focus on the functionality of the system without worrying about its internal workings.
- It provides a clear separation between the tester and the developer, as the tester does not need to know how the system is implemented.
- It helps to uncover defects and errors that are not visible from the system's code or architecture.

Limitations:

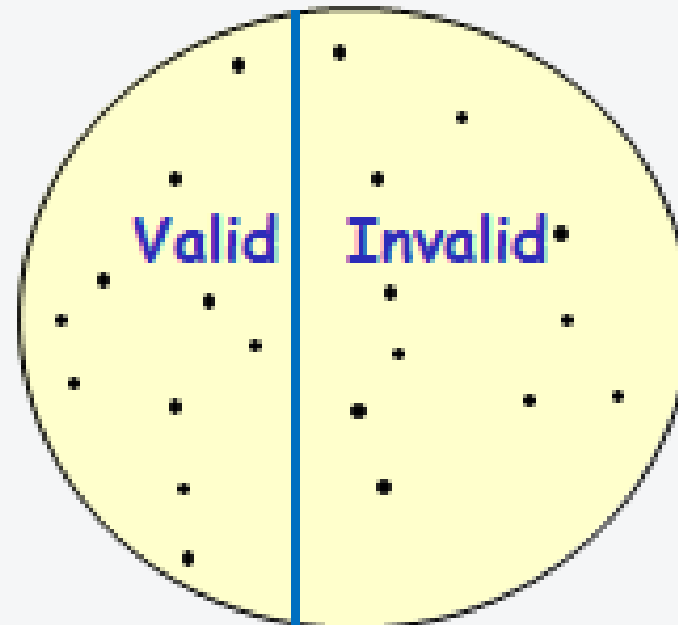
- It may not detect errors that are related to the system's internal workings.
- It may not be as effective in identifying complex defects or bugs.
- It may not be suitable for testing performance or security-related issues.

The techniques used in black-box testing include equivalence partitioning, boundary value analysis, decision table testing, use-case testing etc.,

- Equivalence partitioning is a technique used in software testing that involves dividing a set of test conditions into groups or partitions that are equivalent or similar to each other.
- The idea behind this technique is to minimize the number of test cases that need to be executed while still ensuring that all possible scenarios are covered.

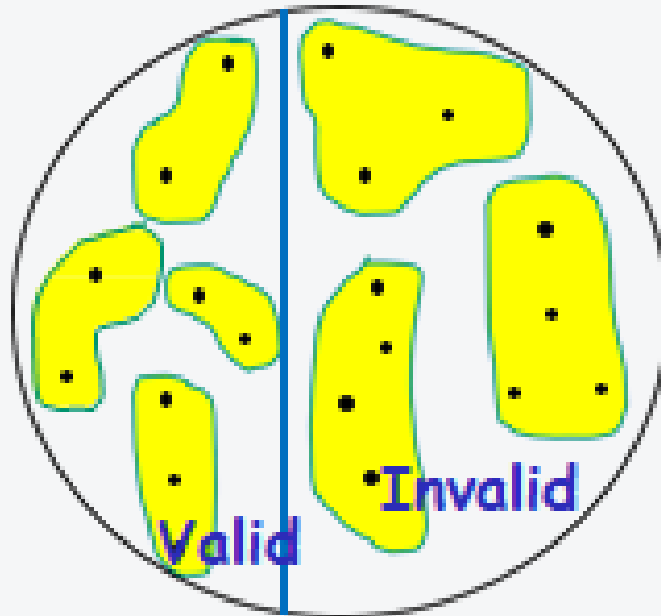
Equivalence Partitioning

- First-level partitioning:
 - Valid vs. Invalid test cases



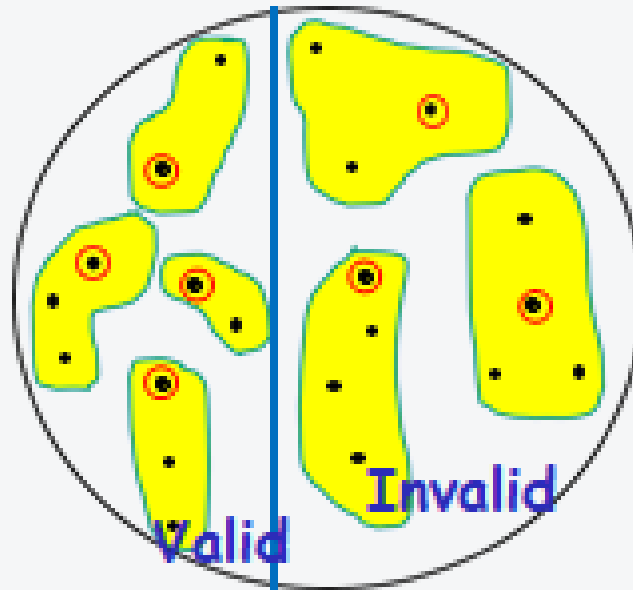
Equivalence Partitioning

- Further partition valid and invalid test cases into equivalence classes



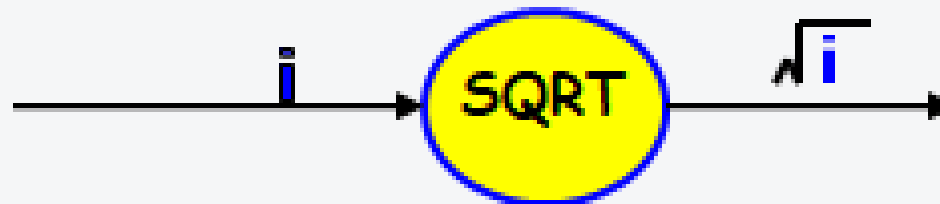
Equivalence Partitioning

- Create a test case for at least one value from each equivalence class



Example

- A program reads an input value in the range of 1 and 5000:
 - Computes the square root of the input number



Example (cont.)

- Three equivalence classes:
 - The set of negative integers,
 - Set of integers in the range of 1 and 5000,
 - Integers larger than 5000.



UQ24CA342A Software Testing and Automation

Equivalence Partitioning



Steps can be followed while applying equivalence partitioning:

1. Identify the input variables of the system to be tested.
2. Divide the input variables into different equivalence classes based on their expected behavior.
3. Develop test cases for each equivalence class.
4. Execute the test cases and analyze the results.
5. Repeat the process for each input variable.

UQ24CA342A Software Testing and Automation

Equivalence Partitioning



Limitations:

1. if the input space is relatively small and well-defined, it may be more efficient to test every input value rather than dividing them into equivalence classes.
2. if the software system being tested is critical or safety-critical, it may be necessary to test every possible input value to ensure that no errors or bugs are present.

UQ24CA342A Software Testing and Automation

Equivalence Partitioning Case Study



User Login Page

- The input fields for the username and password have specific requirements.
- The **username** must be between **6 to 15** characters, and the **password** must be between **8 to 20** characters.
- Equivalence partitioning can be used to identify the valid and invalid input values for each field.

For the username field:

- Valid inputs: username, 123456, qwertyuioplkj
- Invalid inputs: user, 1, qwertyuioplkjzxcvbnm12345

For the password field:

- Valid inputs: password1, Password123, abcdefghi123456789
- Invalid inputs: pass, 1234567, qwertyuioplkjzxcvbnms

UQ24CA342A Software Testing and Automation

Equivalence Partitioning Case Study



Table 5-1. Partitions of Username Field

Invalid	Valid	Invalid
0-5	6-15	>15
Partition 1	Partition 2	Partition 3

Table 5-2. Partitions of Password Field

Invalid	Valid	Invalid
0-7	8-20	>20
Partition 1	Partition 2	Partition 3

Credit Card Payment

The module accepts credit card details such as **card number, expiration date, and security code**. Equivalence partitioning can be used to identify the valid and invalid input values for each field.

For the credit card number field:

- Valid inputs: 16-digit credit card numbers
- Invalid inputs: Less than or more than 16 digits credit card numbers, alphabetic or special characters

For the expiration date field:

- Valid inputs: Any future date in MM/YY format
- Invalid inputs: Any past date, any date with invalid format

UQ24CA342A Software Testing and Automation

Equivalence Partitioning Case Study



Table 5-3. Partitions of Credit Card Number Field

Invalid	Valid	Invalid	Invalid
0-15	16	>16	Non-numbers
Partition 1	Partition 2	Partition 3	Partition 4

Table 5-4. Partitions of Expiration Date Field

Invalid	Valid	Invalid
Past date	Future Date in MM/YY	Invalid format
Partition 1	Partition 2	Partition 3

UQ24CA342A Software Testing and Automation

Classroom Activity: Equivalence Partitioning – Registration Form



A college Online Student Registration Form collects the following information:

Student Name: 3–30 characters

Age: 17–25 years

Email: Must be in a valid email format

Number of Courses: 3–6 courses

Apply Equivalence Partitioning Testing so that the system should accept valid inputs and reject invalid inputs.

1. What is the main purpose of Equivalence Partitioning?

- A. To test every possible input
- B. To divide inputs into groups that are expected to behave similarly
- C. To test only boundary values
- D. To measure performance

2. For an age field accepting 17–25, which set correctly represents the equivalence partitions?

- A. 17, 18, 19
- B. 16, 17, 25, 26
- C. Below 17, 17–25, above 25
- D. 20, 21, 22

3. Which testing technique focuses specifically on dividing the input domain into valid and invalid groups?

- A. Boundary Value Analysis
- B. Equivalence Partitioning
- C. Decision Table Testing
- D. State Transition Testing

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. B
2. C
3. B

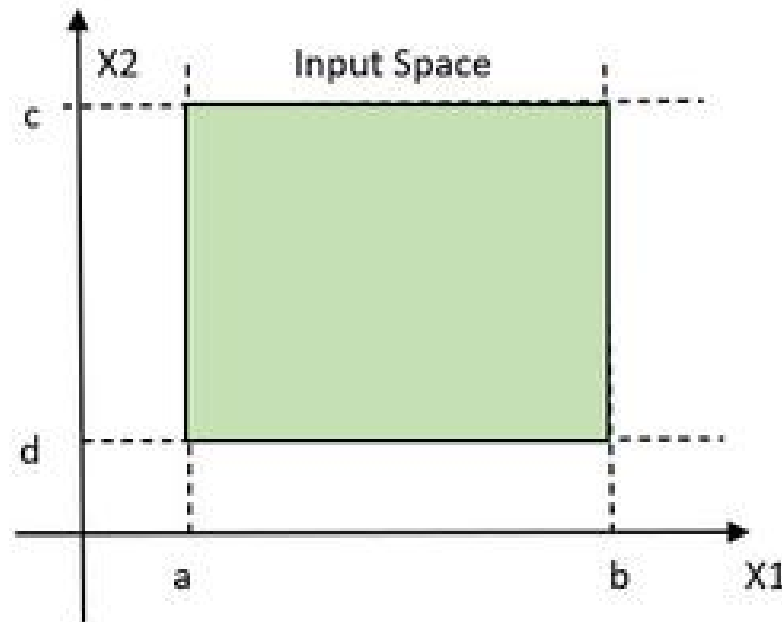
UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

Department of Computer Applications

- Boundary value analysis (BVA) is a software testing technique that is used to identify errors in software by focusing on the boundary conditions of the input domain.
- The input domain is the set of all possible inputs to a software system.



UQ24CA342A Software Testing and Automation

Boundary Value Analysis



The BVA technique involves testing the following values:

Minimum boundary values: These are the smallest values that can be used as input for a particular parameter. For example, if the parameter is the number of items that can be added to a shopping cart, the minimum boundary value would be zero.

Maximum boundary values: These are the largest values that can be used as input for a particular parameter. For example, if the parameter is the number of items that can be added to a shopping cart, the maximum boundary value would be the maximum limit set by the system ie 10.

Just above the minimum boundary values: These are values that are just above the minimum boundary value. For example, if the minimum boundary value for the number of items that can be added to a shopping cart is zero, the just above value would be one.

Just below the maximum boundary values: These are values that are just below the maximum boundary value. For example, if the maximum boundary value for the number of items that can be added to a shopping cart is 10, the just below value would be 9.

By testing these boundary values, testers can identify any issues that may exist in the software. For example, if the system is designed to allow a maximum of 10 items to be added to a shopping cart, testing with just above and just below values may reveal issues with how the system handles values that exceed this limit.

Limitations (when BVA not applicable):

- 1) when testing scenarios where inputs are not restricted by boundaries or when the boundaries themselves are not well-defined or may change frequently

Ex:

- 2) when inputs have a large number of variables and dimensions, as identifying and testing every possible boundary condition can be time-consuming and resource-intensive.

Ex:

UQ24CA342A Software Testing and Automation

Boundary Value Analysis



Age Validation: Suppose you are testing a form that requires users to enter their age.

The system allows users who are 18 years or older to register.

The input field for age has specific requirements. The user's age must be between 18 to 60.

Test cases can be designed for these boundary values to check if the system handles them correctly.

- Test case 1: Enter age as 17: This should be rejected with an appropriate error message.
- Test case 2: Enter age as 18: This should be accepted.
- Test case 3: Enter age as 60: This should be accepted.
- Test case 4: Enter age as 61: This should be rejected with an appropriate error message.

UQ24CA342A Software Testing and Automation

Boundary Value Analysis



File Size Validation: Suppose you are testing a file upload feature on a website. The feature allows users to upload files of up to 10 MB in size.

Test cases can be designed for these

boundary values to check if the system handles them correctly.

- Test case 1: Upload a file with 0 MB size: This should be rejected with an appropriate error message.
- Test case 2: Upload a file with 10 MB size: This should be accepted.
- Test case 3: Upload a file with 11 MB size: This should be rejected with an appropriate error message.

UQ24CA342A Software Testing and Automation

Classroom Activity on Boundary Value Analysis



A college Online Course Registration System. During registration, the system asks students to enter the number of courses they want to register for. A student can register for a minimum of 3 courses and a maximum of 6 courses in one semester.

Questions:

- 1) Identify the minimum and maximum boundaries.
- 2) Identify the values that should be tested around each boundary.
- 3) Prepare the test cases ****
- 4) Identify which test cases should pass and which should fail.
- 5) Explain why boundary values are important for testing this system.

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. What is the primary purpose of Boundary Value Analysis?

- A. To test program performance
- B. To test values at and around input boundaries
- C. To test source code structure
- D. To test database connectivity

2. For an input range of 10 to 50, which value is a boundary value?

- A. 30
- B. 35
- C. 10
- D. 25

3. For a valid range of 25 to 75, which set contains only boundary-related test values?

- A. 30, 40, 50, 60
- B. 24, 25, 26, 74, 75, 76
- C. 25, 35, 45, 55, 65
- D. 20, 30, 40, 50, 60, 70

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. B
2. C
3. B

UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

Department of Computer Applications

- Decision table testing, also known as cause-effect graphing, is a black-box testing technique that is used to test systems that have multiple inputs and outputs, where the output is dependent on the input and a set of rules.
- It is often used in complex systems where there are multiple combinations of inputs that can be tested.
- The process of decision table testing involves creating a table that shows all possible combinations of inputs and their corresponding outputs.
- Each combination of inputs is referred to as a rule.
- The table also includes additional columns to identify the conditions that must be met for each rule to be executed, as well as the actions that must be taken as a result of each rule.

	Rule1	Rule2	Rule3	Rule4
Condition1	Yes	Yes	No	No
Condition2	Yes	X	No	X
Condition3	No	Yes	No	X
Condition4	No	Yes	No	Yes
Action1	Yes	Yes	No	No
Action2	No	No	Yes	No
Action3	No	No	No	Yes

UQ24CA342A Software Testing and Automation

Decision Table Testing Example

This table shows the combinations of conditions (Yes/No) and the actions to be taken for each rule.

Y = Yes (Condition is true) N = No (Condition is false) X = Action to be taken

CONDITIONS		R1	R2	R3	R4	R5	R6	R7	R8
	Printer does not print	Y	Y	Y	Y	N	N	N	N
	A red light is flashing	Y	Y	N	N	Y	Y	N	N
	Printer is unrecognized	Y	N	Y	N	Y	N	Y	N
ACTIONS	Check the power cable			X				X	X
	Check the printer-computer cable	X	X					X	X
	Ensure printer software is installed	X	X		X		X	X	X
	Check/replace ink	X	X			X	X	X	X
	Check for paper jam		X		X			X	X

How to read the table

- 1 Each column (R1 to R8) is a RULE.
- 2 The top part (green) shows the condition values (Y/N) for that rule.
- 3 The bottom part (yellow) shows the actions (X) to be performed when the conditions in that column are true.
- 4 Blank cells mean the action is not required for that rule.

Example: For Rule R7

Conditions are: Printer does not print = N
A red light is flashing = N
Printer is unrecognized = Y

Actions required (see X in column R7):

- Check the power cable
- Check the printer-computer cable
- Ensure printer software is installed
- Check/replace ink
- Check for paper jam

Interpretation

If the printer is unrecognized but not printing and no red light is flashing, perform all the actions marked X in R7.

Limitations:

- 1) when there are only a few conditions to be tested as it can be time-consuming to create the table and may not provide significant benefits.
- 2) It may also be less effective when there are dependencies between the conditions and the order of testing matters.

UQ24CA342A Software Testing and Automation

Decision Table Testing Case Study



E-shop Discounts:

An e-commerce website sells various products, and it has different discount offers for each product. The discounts depend on the product type, product price, and customer's age.

For electronics, if(product price < \$500) then the discount = 5% for customers below 18 and 10% for customers who are 18 and over.

If(product price >= \$500) then the discount is 10% for customers below 18 and 20% for customers who are 18 and over.

For clothing, if(product price < \$100), then the discount is 5% for customers below 18 and 10% for customers who are 18 and over.

If(product price >= \$100), then the discount is 15% for customers below 18 and 20% for customers who are 18 and over.

UQ24CA342A Software Testing and Automation

Decision Table Testing Case Study



Table 5-5. Decision Table **Conditions** **Actions**

Product Type	Product Price	Customer Age	Discount Offer
Electronics	>= \$500	< 18	10% off
Electronics	>= \$500	>= 18	20% off
Electronics	< \$500	< 18	5% off
Electronics	< \$500	>= 18	10% off
Clothing	>= \$100	< 18	15% off
Clothing	>= \$100	>= 18	20% off
Clothing	< \$100	< 18	5% off
Clothing	< \$100	>= 18	10% off

UQ24CA342A Software Testing and Automation

Decision Table Testing Case Study



Transportation Price:

A transportation company provides transportation services for customers, and it charges based on the distance and the number of passengers.

The pricing depends on the type of transportation, which can be either a taxi or a bus.

Conditions:

- A taxi with one or two passengers charges \$5, \$10, and \$20 for a distance of 0–5, 5–10, and 10+ km, respectively.
- A taxi with three or four passengers charges \$10, \$20, and \$40 for a distance of 0–5, 5–10, and 10+ km, respectively.
- A bus with 1–30 passengers charges \$50, \$100, and \$200 for a distance of 0–10, 10–50, and 50+ km, respectively.

Table 5-6. Decision Table **Conditions** **Actions**

Type of Transportation	Number of Passengers	Distance	Price
Taxi	1–2	0–5	\$5
Taxi	1–2	5–10	\$10
Taxi	1–2	10+	\$20
Taxi	3–4	0–5	\$10
Taxi	3–4	5–10	\$20
Taxi	3–4	10+	\$40
Bus	1–30	0–10	\$50
Bus	1–30	10–50	\$100
Bus	1–30	50+	\$200

UQ24CA342A Software Testing and Automation

Classroom Activity on Decision Table Testing

Create Decision Tree for ATM Cash Withdrawal Scenario

An ATM allows a customer to withdraw money based on the following conditions:

- The card is valid.
- The PIN is correct.
- The account has sufficient balance.
- The ATM has sufficient cash.

The system follows these rules:

- If the card is invalid → **Transaction Rejected.**
- If the card is valid but the PIN is incorrect → **Transaction Rejected.**
- If the card and PIN are valid but the account has insufficient balance → **Transaction Rejected.**
- If the account has sufficient balance but the ATM does not have enough cash → **Transaction Rejected.**
- If all conditions are satisfied → **Cash Dispensed.**

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. What is the main purpose of Decision Tree Testing?

- A. To test program performance
- B. To represent and test different combinations of conditions and decisions
- C. To test database structure
- D. To measure code coverage

2. In a decision tree, the branches generally represent:

- A. Test case IDs
- B. Input/output files
- C. Possible outcomes of conditions
- D. Program modules

3. Consider the rule: "If the password is correct, login is successful; otherwise, login fails." How many possible outcomes are there?

- A. 1
- B. 2
- C. 3
- D. 4

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. B
2. C
3. B

UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

Department of Computer Applications

- Pairwise testing, also known as all-pairs testing, is a technique used in software testing to ensure that all possible combinations of input parameters are tested at least once.
- It is a combinatorial testing method that reduces the number of test cases required to cover all possible combinations of inputs, while still providing a high level of test coverage.
- Pairwise testing works by creating test cases that cover all possible pairs of values for each input parameter.

- For example, if a system has three input parameters, each with four possible values, there would be a total of 64 possible combinations ($4 \times 4 \times 4$).
- With pairwise testing, however, it is possible to reduce the number of test cases required to cover all possible pairs of values to just 24, significantly reducing the time and effort required for testing.
- Pairwise testing can be particularly useful in situations where there are a large number of input parameters or where the interactions between input parameters are complex and difficult to predict

UQ24CA342A Software Testing and Automation

Pairwise Testing Case Study



Example: web form that asks for a user's personal information, including their first name, last name, email address, phone number, and country.

The form has the following input fields:

- First Name (text field, max 50 characters)
- Last Name (text field, max 50 characters)
- Email Address (text field, max 100 characters)
- Phone Number (text field, max 20 characters)
- Country (drop-down list of 10 countries)

UQ24CA342A Software Testing and Automation

Pairwise Testing Case Study



Table 5-9. All Users

First Name	Last Name	Email Address	Phone Number	Country
Alex	Smith	alex@test.com	1234567890	USA
Bob	Jones	bob@test.com	0987654321	Canada
Charlie	Brown	charlie@test.com	5555555555	UK
Dave	Kim	dave@test.com	1112223333	Japan
Emma	Lee	emma@test.com	4445556666	Australia

UQ24CA342A Software Testing and Automation

Pairwise Testing Case Study



Next, we group the parameters into pairs and generate all possible combinations, like so:

- First Name, Last Name
- First Name, Email Address
- First Name, Phone Number
- First Name, Country
- Last Name, Email Address
- Last Name, Phone Number
- Last Name, Country
- Email Address, Phone Number
- Email Address, Country
- Phone Number, Country

UQ24CA342A Software Testing and Automation

Pairwise Testing Case Study



For each pair, we select test cases that cover all possible combinations of values.

Ex: **First Name and Last Name**, we can select the following test cases:

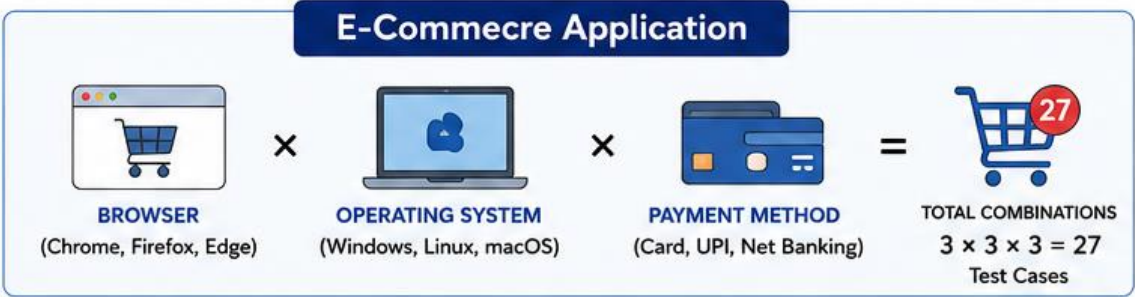
- Enter **valid** values for both fields (e.g., Alex and Smith).
- Enter the **maximum** allowed number of characters for each field (50 characters).
- Enter an **empty value** for one field and a **valid value** for the other (e.g., leave First Name blank and enter Smith for Last Name).
- Enter an **invalid value** for one field and a **valid value** for the other (e.g., enter 123 for First Name and Smith for Last Name).
- Enter **non-ASCII characters** for both fields (e.g., Álex and Smith).

We repeat this process for each pair of input parameters, until we have selected test cases that cover all possible combinations of values.

UQ24CA342A Software Testing and Automation

Pairwise Testing Case Study

ALL POSSIBLE COMBINATIONS (Too Many Combinations!)



Exhaustive testing = $3 \times 3 \times 3 = 27$ test cases
Too many test cases → Time consuming and expensive

EXAMPLE SCENARIO

An e-commerce application has 3 parameters with the following values:

Parameter	Values
Browser	Chrome, Firefox, Edge
Operating System (OS)	Windows, Linux, macOS
Payment Method	Card, UPI, Net Banking



Pairwise testing selects a smaller set of test cases so that every possible pair of values appears at least once.

PAIRWISE TEST SET (Fewer Test Cases – Smarter Choice)

Test Case ID	Browser	Operating System	Payment Method
TC01	Chrome	Windows	Card
TC02	Chrome	Linux	UPI
TC03	Chrome	macOS	Net Banking
TC04	Firefox	Windows	UPI
TC05	Firefox	Linux	Net Banking
TC06	Firefox	macOS	Card
TC07	Edge	Windows	Net Banking
TC08	Edge	Linux	Card
TC09	Edge	macOS	UPI

Only 9 test cases instead of 27!

PAIRS COVERED AT LEAST ONCE

Browser – OS Pairs
Chrome – Windows
Chrome – Linux
Chrome – macOS
Firefox – Windows
Firefox – Linux
Firefox – macOS
Edge – Windows
Edge – Linux
Edge – macOS
✓ All covered

Browser – Payment Pairs
Chrome – Card
Chrome – UPI
Chrome – Net Banking
Firefox – Card
Firefox – UPI
Firefox – Net Banking
Edge – Card
Edge – UPI
Edge – Net Banking
✓ All covered

OS – Payment Pairs
Windows – Card
Windows – UPI
Windows – Net Banking
Linux – Card
Linux – UPI
Linux – Net Banking
macOS – Card
macOS – UPI
macOS – Net Banking
✓ All covered

Apply Pairwise Testing for a mobile banking application needs to be tested on different combinations of:

Parameter	Values
Mobile OS	Android, iOS
Network	Wi-Fi, 4G, 5G
Login Method	Password, Fingerprint, Face ID

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. What is the main objective of Pairwise Testing?

- A. Test every possible combination
- B. Test every individual input separately
- C. Ensure every pair of parameter values is tested at least once
- D. Test only boundary values

2. Which of the following is a major advantage of Pairwise Testing?

- A. It tests all possible combinations
- B. It reduces the number of test cases
- C. It eliminates the need for functional testing
- D. It tests only valid inputs

3. In Pairwise Testing, what does a "pair" refer to?

- A. Two test cases
- B. Two defects
- C. Two parameter values from different parameters
- D. Two users

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. C
2. B
3. C

UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

Department of Computer Applications

- Use-case testing is a software testing technique that evaluates the software's ability to meet the end users' requirements.
- A use case is a specific scenario that describes how a user interacts with the software system.
- Use-case testing involves creating test cases based on these scenarios to verify that the system behaves as expected.

Benefits of use-case testing:

Improved software quality: Use-case testing helps to ensure that the software meets the end users' requirements, resulting in higher-quality software.

Better test coverage: Use-case testing ensures that all possible scenarios are tested, leading to better test coverage.

Easy to understand: Use cases are easy to understand, making it easier for stakeholders to provide feedback and review the software.

Efficient testing: Use-case testing is efficient because it focuses on the most important scenarios, reducing the time and effort required for testing.

UQ24CA342A Software Testing and Automation

Use-Case Testing Case Study



For example, **consider a banking application**

- A use case for this application could be withdrawing cash from an ATM.
- Test cases for this use case might include verifying that the ATM is functioning correctly, the user is authenticated, the account balance is updated, and the user receives the requested cash amount.
- By testing the use case, we can verify that the banking application meets the user's requirements and functions as intended.
- This approach is also helpful when the system is expected to undergo changes frequently

Limitations :

- Use-case testing may not be suitable when the system is relatively simple and the requirements are straightforward.
- It may also not be appropriate when there is not enough information available about the user's perspective and the system's use cases.
- Additionally, use-case testing can be time-consuming and may require significant effort to design and execute the test cases

UQ24CA342A Software Testing and Automation

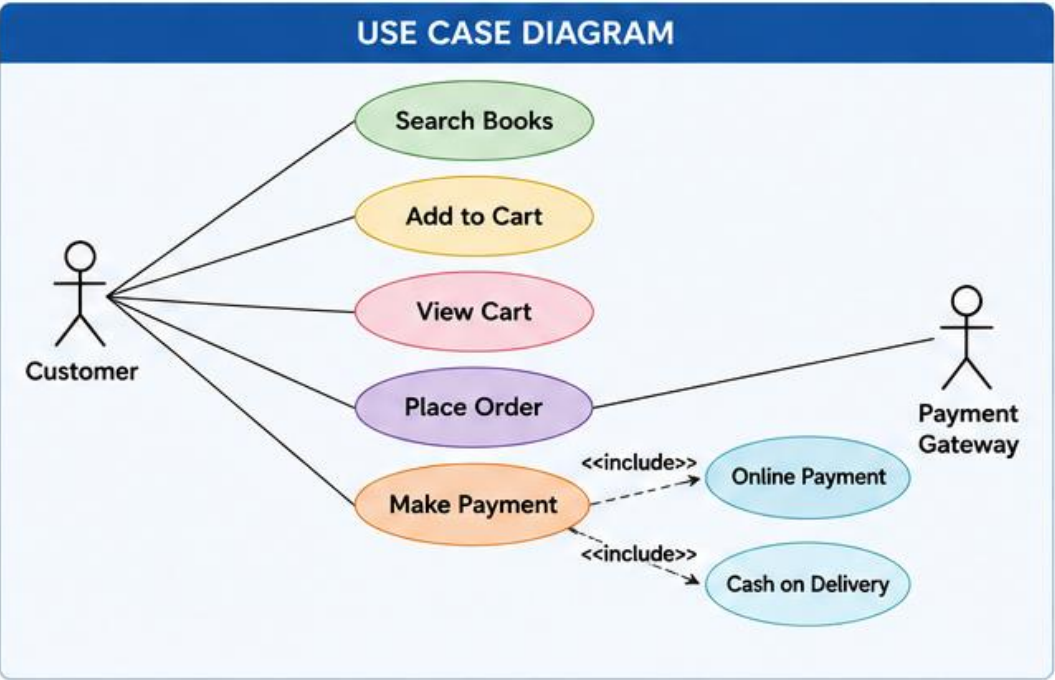
Use-Case Testing Example

EXAMPLE APPLICATION

Online Book Store

A customer can search books, add to cart and place an order.





USE CASE DESCRIPTIONS (Brief)	
Use Case	Description
Search Books	Customer searches for books by title, author or category.
Add to Cart	Customer adds a selected book to the shopping cart.
View Cart	Customer views the list of books in the cart.
Place Order	Customer confirms the order with delivery details.
Make Payment	Customer chooses a payment method and makes payment.
Online Payment	Payment is made using card/ UPI/Net Banking.
Cash on Delivery	Customer pays cash at the time of delivery.

UQ24CA342A Software Testing and Automation

Classroom Activity: Use Case Testing – Online Railway Ticket Booking



Draw Use-case diagram & Apply Use-case Testing using test cases

Consider an Online Railway Ticket Booking System. A passenger can:

- Search for trains
- Select a train
- Enter passenger details
- Select a seat/class
- Make payment
- Receive the ticket

The system should also handle payment failure, unavailable seats, and invalid passenger details.

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. What is the main purpose of Use Case Testing?

- A. To test individual lines of code
- B. To verify system functionality from the user's perspective
- C. To test database performance
- D. To test only the user interface

2. In a use case, the person or system that interacts with the application is called:

- A. Object
- B. Actor
- C. Class
- D. Component

3. Which of the following should be tested in Use Case Testing?

- A. Only the main successful flow
- B. Only invalid inputs
- C. Main, alternative, and exception flows
- D. Only boundary values

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. B
2. B
3. C

UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

Department of Computer Applications

- State transition testing is a black-box testing technique used to test the functionality of a system or software that involves the transition of the system from one state to another.
- In this technique, the behavior of the system is analyzed based on different input conditions and the resulting transitions from one state to another.
- The testing process involves the identification of all possible states and the transitions between these states.

The process of state transition testing involves the following steps:

1. **Identify the states of the system:** The first step is to identify all the states of the system. A state is a condition or mode in which the system operates.
2. **Identify the events that cause transitions:** The next step is to identify all the events that cause the system to transition from one state to another.
3. **Create a state transition diagram:** A state transition diagram is created to illustrate the various states of the system and the transitions between them.
4. **Design test cases:** Test cases are designed to cover each transition and the associated behavior.
5. **Execute test cases:** The test cases are executed to verify that the system is functioning as expected.

UQ24CA342A Software Testing and Automation

State Transition Testing Case Study



Traffic Light System:

Suppose you are testing a traffic light system where the light turns from red to green to yellow to red, based on a timer. In this example, the traffic light system has three states: Red, Green, Yellow.

State Transition Table

Table 5-7 shows the traffic light states.

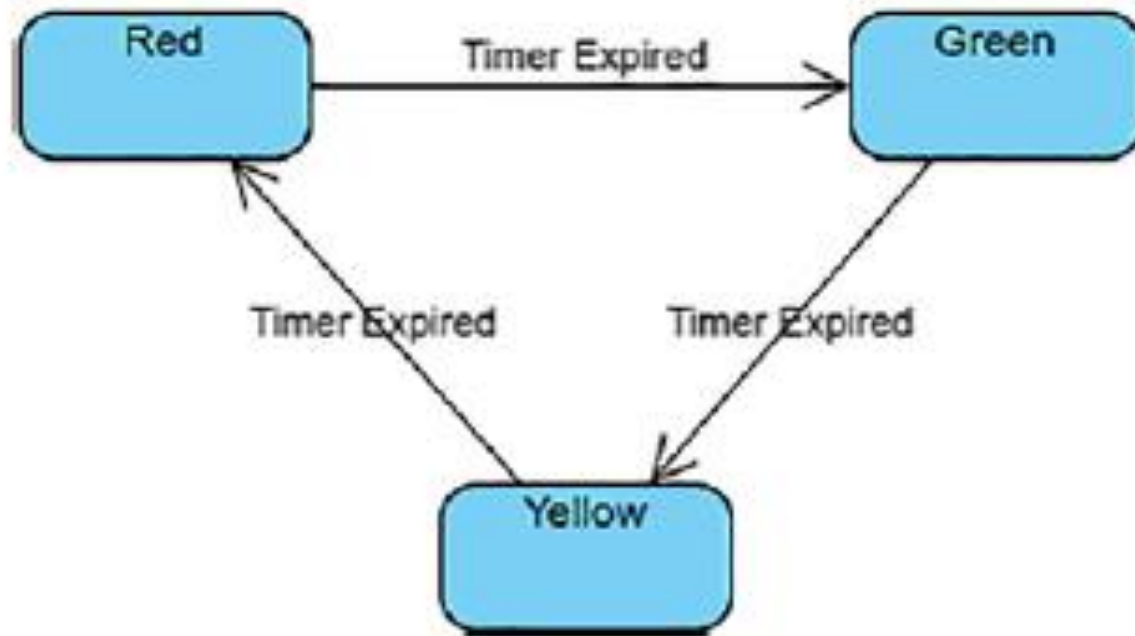
Table 5-7. Traffic Light System States

Current State	Event	Next State
Red	Timer expired	Green
Green	Timer expired	Yellow
Yellow	Timer expired	Red

UQ24CA342A Software Testing and Automation

State Transition Testing Case Study

State Transition Diagram



Traffic light state transition diagram

UQ24CA342A Software Testing and Automation

State Transition Testing Case Study



Online shopping cart:

- In this example, application has four states: Empty Cart, One Item, Two Items, and Three Items.
- There is also a special state called Checkout that represents the process of checking out and paying for items in the cart.
- The state transition table lists all of the possible events that can occur in each state, along with the resulting next state.
- For example, if the current state is Empty Cart and the Add Item event occurs, the next state will be One Item. If the current state is One Item and the Proceed to Checkout event occurs, the next state will be Checkout.

UQ24CA342A Software Testing and Automation

State Transition Testing Case Study

State Transition Table:

Table 5-8. Shopping Cart States

Current State	Event	Next State
Empty Cart	Add Item	One Item
Empty Cart	Remove Item	Error
One Item	Add Item	Two Items
One Item	Remove Item	Empty Cart
One Item	Proceed to Checkout	Checkout
Two Items	Add Item	Three Items
Two Items	Remove Item	One Item
Two Items	Proceed to Checkout	Checkout
Three Items	Remove Item	Two Items
Three Items	Proceed to Checkout	Checkout
Checkout	Cancel	Empty Cart
Checkout	Confirm Payment	Thank You

UQ24CA342A Software Testing and Automation

State Transition Testing

State Transition Diagram

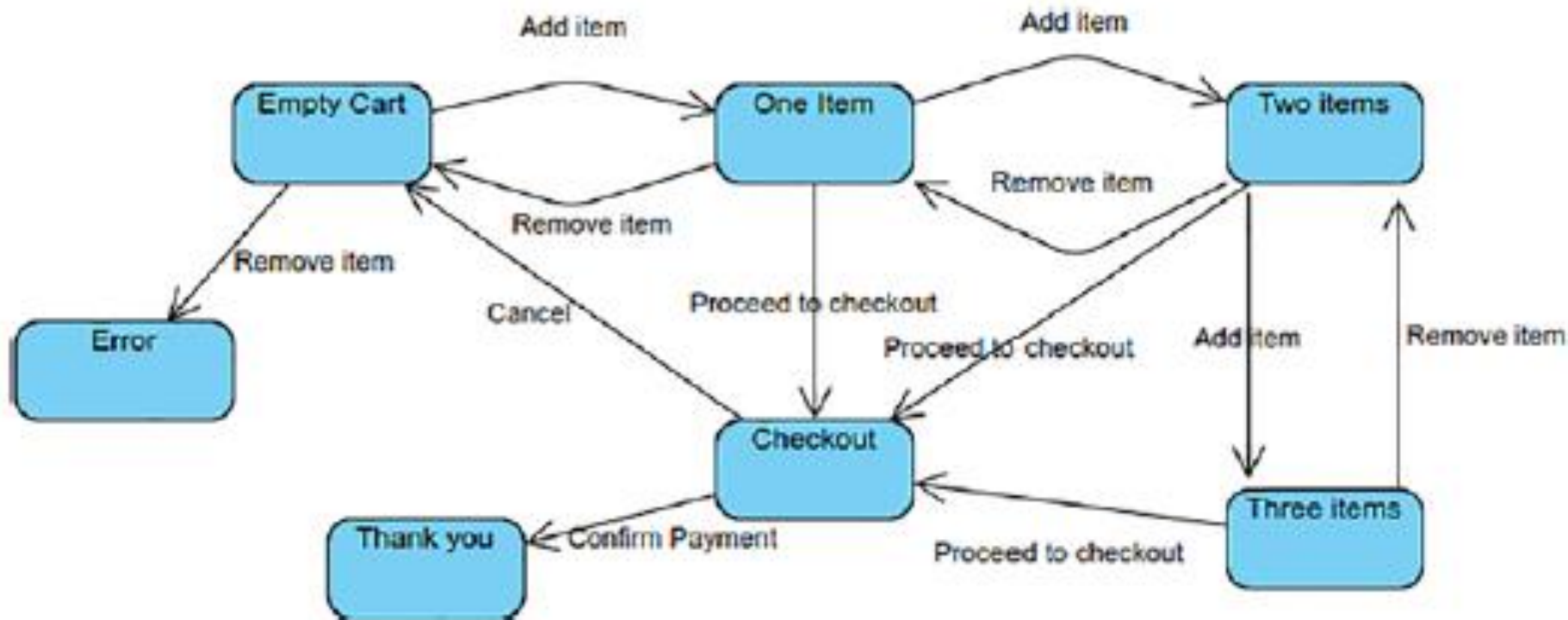


Figure 5-2. Shopping cart state transition diagram

UQ24CA342A Software Testing and Automation

State Transition Testing



Limitations:

- Applications that do not have clear states or modes may not benefit from this technique.
- Applications that have a limited number of states or have simple transitions may not require state transition testing.

UQ24CA342A Software Testing and Automation

State Transition Testing Example

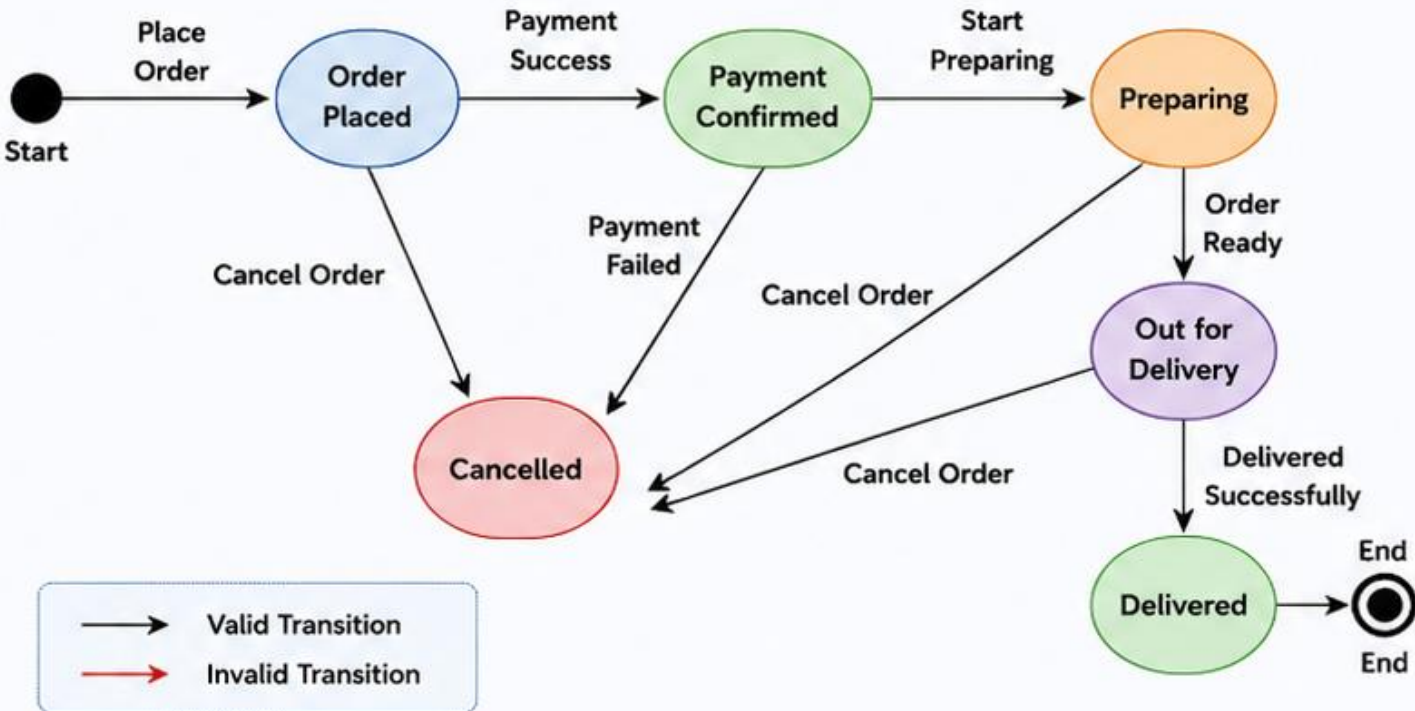
REAL TIME EXAMPLE

Online Food Delivery Application – Order Life Cycle



	Order Placed Customer places the order
	Payment Confirmed Payment is successful
	Preparing Restaurant is preparing the order
	Out for Delivery Rider is on the way
	Delivered Order is delivered to customer
	Cancelled Order is cancelled

STATE TRANSITION DIAGRAM



Apply & Draw State Transition Testing for ATM Card

An ATM system manages a customer's card through different states.

The card can be in these states:

- Card Inserted
- PIN Verification
- Authenticated
- Transaction
- Card Ejected
- Card Blocked

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. What is the main purpose of State Transition Testing?

- A. To test input boundaries
- B. To verify how a system changes from one state to another
- C. To test database performance
- D. To test program code structure

2. In State Transition Testing, a "state" represents:

- A. A test case number
- B. The current condition or status of the system
- C. A software defect
- D. An input value only

3. What causes a transition from one state to another?

- A. Event or action
- B. Test report
- C. Test data size
- D. Program documentation

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. B
2. B
3. A

UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

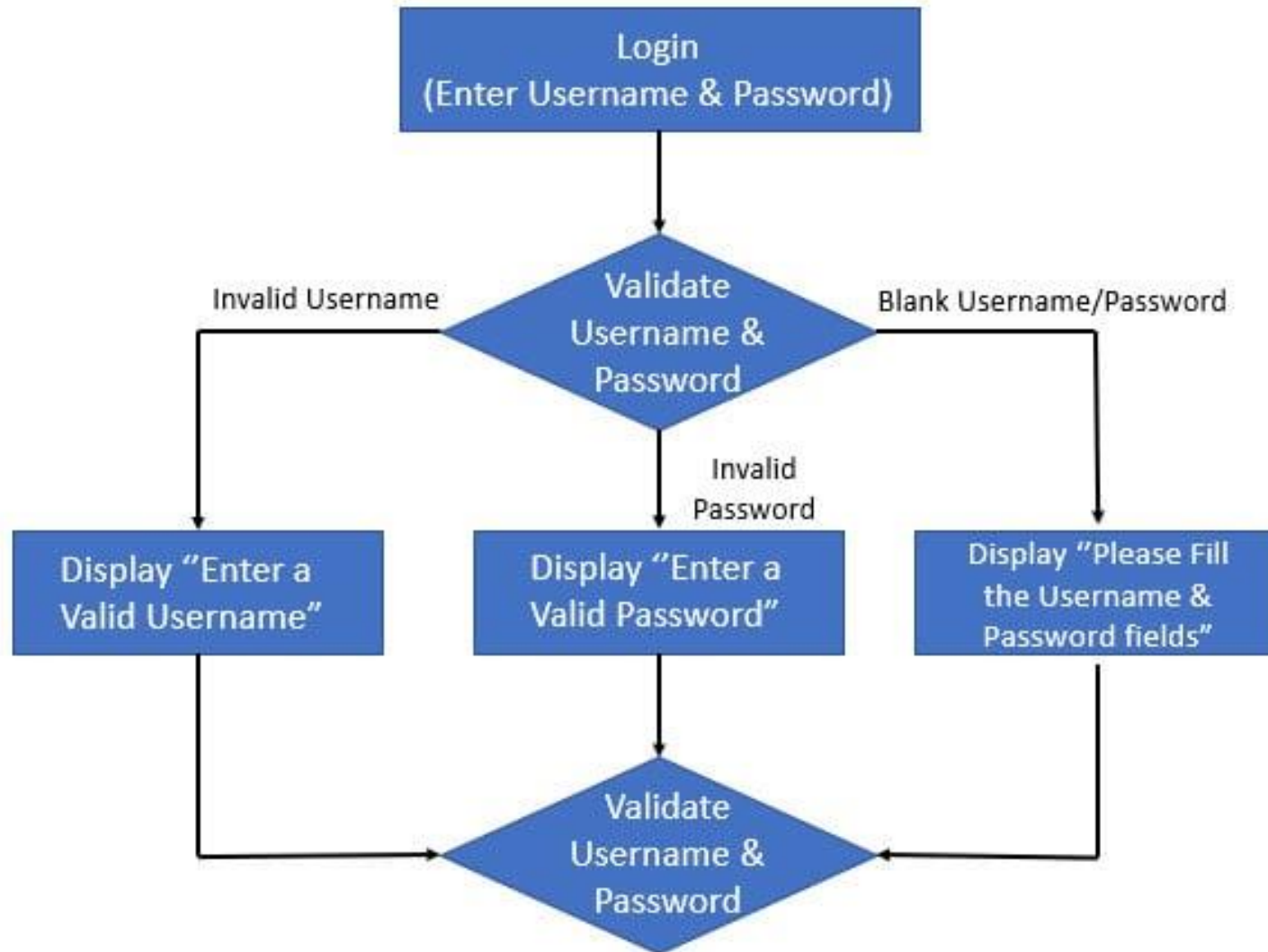
Department of Computer Applications

- Branch coverage is a white-box testing technique that evaluates whether all the branches in the program source code are executed at least once.
- The primary objective of branch coverage testing is to determine whether each decision point in the code has been tested.
- The decision point is a code statement that determines which of two paths to take.
- Branch coverage is measured by the percentage of the total number of decision points executed during the testing process.

- For example, consider a code block with an if-else statement.
- The if-else statement has two branches: one that is executed if the condition is true and the other if the condition is false.
- To test branch coverage, create test cases that execute both the true and false branches at least once..

Benefits of branch coverage testing:

- It helps to identify the potential faults in the code by testing all possible paths.
- It ensures that the program's control flow is working as expected, reducing the risk of runtime errors.
- It provides metrics to measure the testing effectiveness and code quality.
- It provides confidence in the code quality, helping to reduce the cost of fixing defects.



UQ24CA342A Software Testing and Automation

Branch Coverage Explanation

```
read username  
read password  
if count (username) < 8  
print “enter a valid username”  
endif  
if count (password) < 5  
print “enter a valid password”  
endif
```

```
if count(username & password) < 1  
print “please fill the username & password fields”  
endif  
else  
print “login successfully”
```

- In this example, the login page validation has three possible outcomes.
- When the login credentials are validated for the correctness, there rise three possible functional flows.
- If the User fails to enter a valid username, then the first conditional loop will be executed, if the user fails to enter a valid password, the second flow will be executed and if both the credential fields are not passed with any values, the third condition will be carried out.
- Branch Coverage Testing checks all the potential branching for functional looping statements.

```
1. def add_numbers(a, b):  
2.     if a > 0 and b > 0:  
3.         result = a + b  
4.     else:  
5.         result = 0  
6.     return result
```

To test the branch coverage of this code, write test cases that execute each branch at least once.

Table shows the minimum set of test cases to achieve 100 percent branch coverage.

Branch Coverage (Add Numbers)

Test Case	A	b	Result
1	3	4	7
2	0	2	0

Branch Coverage Example 1

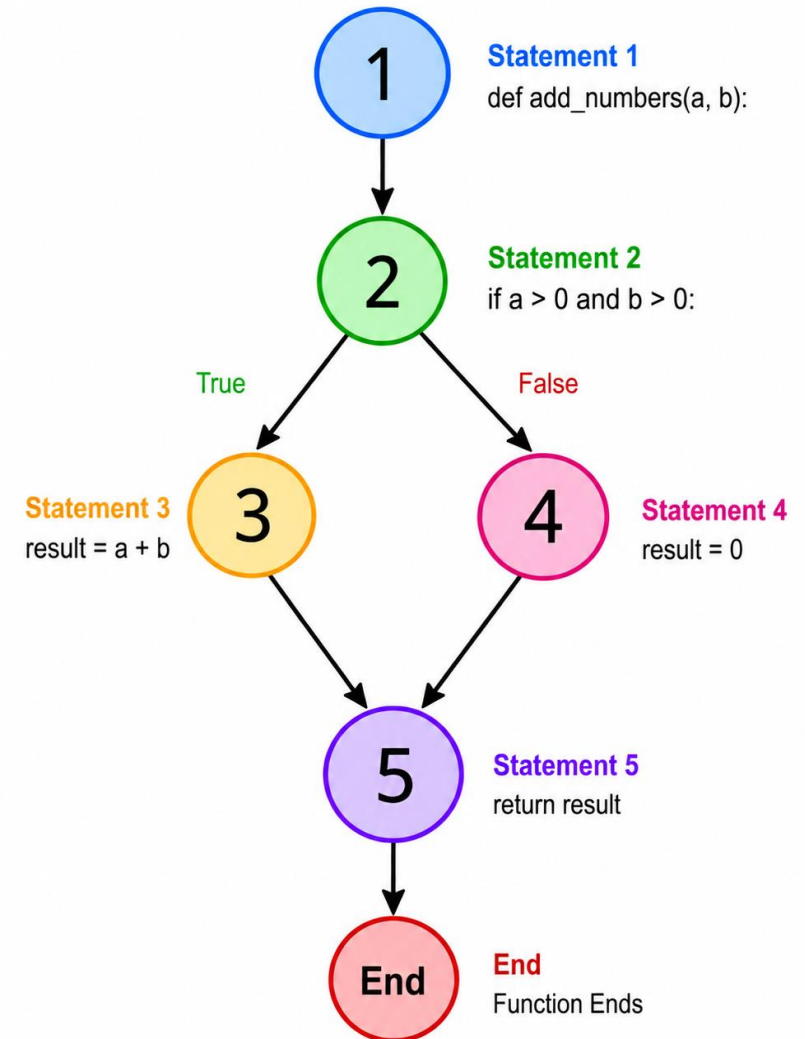
```
def add_numbers(a,b):  #statement 1
    if a>0 and b>0:    #statement 2
        result = a+b   #statement 3
    else:               #statement 4
        result = 0      #statement 4
    return result       #statement 5
```

Test Case 1: a=2, b=2

Path: Statement 1 → Statement 2 (condition is True)
→ Statement 3

Statements Executed: 1, 2, 3.

Coverage so far: $3/5 = 60\%$



Control Flow Graph/ Program Graph

Branch Coverage Example 1

```
def add_numbers(a,b):  #statement 1
    if a>0 and b>0:    #statement 2
        result = a+b   #statement 3
    else:
        result = 0      #statement 4
    return result       #statement 5
```

Test Case 2: a=0, b=-2

Path: Statement 1 → Statement 2 (condition is False) → Statement 4 → Statement 5

Statements Executed: 1, 2, 4, 5

Coverage so far: $4/5 = 90\%$

Totally: 1,2,3,4,5 → $5/5 = 100\%$

```
def max_of_two_numbers(x, y):  
    if x > y:  
        return x  
    else:  
        return y
```

To achieve 100 percent branch coverage for this function, test both possible outcomes of the if statement: when x is greater than y, and when x is not greater than y.

Branch Coverage (Max of Two Numbers)

Test Case	a	b	Result
1	3	4	4
2	3	2	3

Branch Coverage Example 2

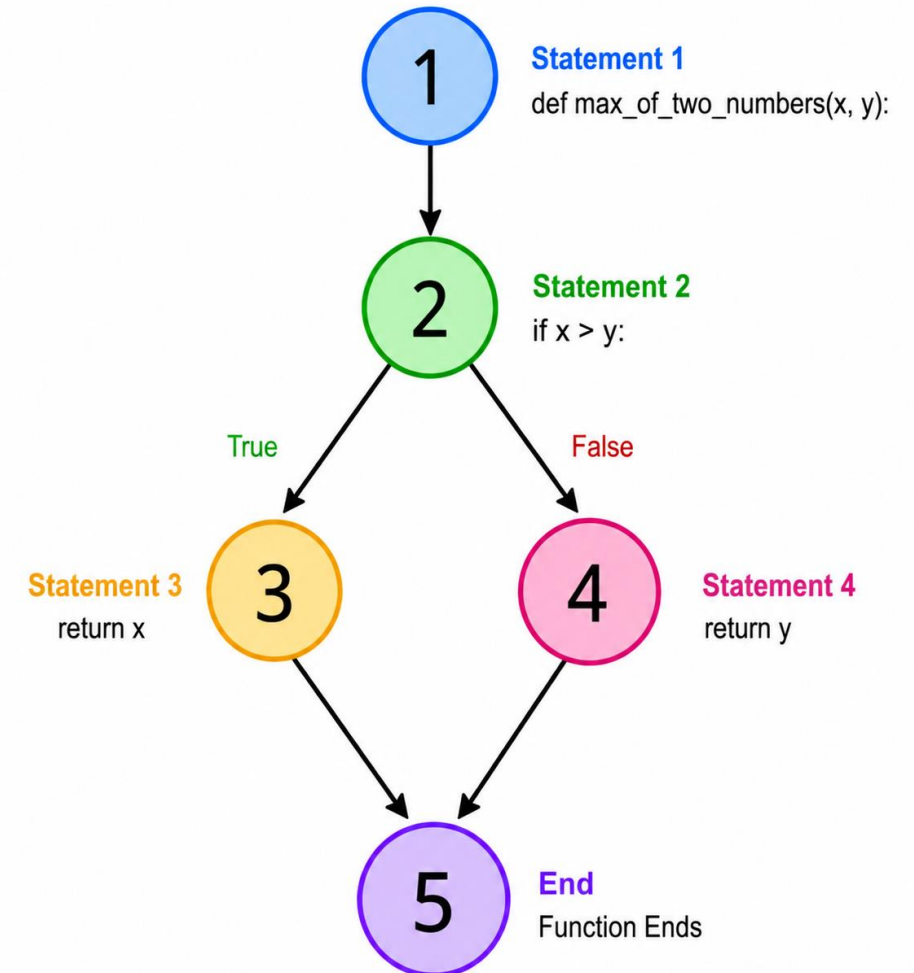
```
def max_of_two_numbers(x,y):  #statement 1
    if x>y:                  #statement 2
        return x             #statement 3
    else:                    #statement 4
        return y
```

Test Case 1: x=4, y=3

Path: Statement 1 → Statement 2 (condition is True)
→ Statement 3

Statements Executed: 1, 2, 3

Coverage so far: $3/4 = 90\%$



Branch Coverage Example 2

```
def max_of_two_numbers(x,y): #statement 1
    if x>y:                  #statement 2
        return x             #statement 3
    else:
        return y             #statement 4
```

Test Case 1: x=3, y=4

Path: Statement 1 → Statement 2 (condition is False) → Statement 4

Statements Executed: 1, 2, 4

Coverage so far: $3/4 = 90\%$

Totally: 1,2,3,4 $4/4=100\%$

Conclusion:

Achieving branch coverage may not be sufficient to test all possible code paths. Therefore, additional testing techniques such as path coverage and condition coverage may also be necessary to ensure thorough testing.

UQ24CA342A Software Testing and Automation

Classroom Activity on Branch Coverage Testing



Write Branch Coverage testing test cases for below code:

```
def classify_number(x):  
    if x > 0:  
        print("Positive")  
    elif x == 0:  
        print("Zero")  
    else:  
        print("Negative")
```

1. Branch Coverage is mainly associated with which type of testing?

- A) Black-box testing
- B) White-box testing
- C) Acceptance testing
- D) Performance testing

2. For an if-else statement, achieving 100% Branch Coverage generally requires:

- A) Testing only the if condition as True
- B) Testing only the else condition
- C) Executing both the True and False branches
- D) Executing the if statement twice with the same input

3. What does 100% Branch Coverage indicate?

- A) Every possible input has been tested
- B) Every executable statement has been executed
- C) Every identified branch has been executed at least once
- D) The program contains no defects

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. B
2. C
3. C

UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

Department of Computer Applications

- White-box testing is a software testing technique that is used to test the internal workings of an application or system.
- In white-box testing, the tester has knowledge of the internal structure and implementation of the software being tested.
- Here are some white-box testing techniques: Statement Coverage, Branch Coverage, Path Coverage, Condition Coverage, Decision Coverage, Multiple Condition Coverage, Modified Condition/Decision Coverage, Loop Testing, Data Flow Testing

Statement coverage

- Statement coverage is a type of white-box testing technique that is used to determine **how many statements in the source code** of a software system have been executed during the testing process.
- The goal of statement coverage is to ensure that **each statement in the code has been executed at least once** during the testing process.
- The goal of statement coverage is to achieve 100 percent statement coverage, which means that every statement in the code has been executed at least once.

Statement coverage has some limitations.

- For example, achieving 100 percent statement coverage does not guarantee that the software system is completely free of defects. It is possible to have a software system with 100 percent statement coverage that still contains defects.
- statement coverage can be a time-consuming process, especially for large software systems.

Consider a simple python function with if condition

```
def largest(a, b):  
    if a > b:      # Statement 1  
        largest = a  # Statement 2  
    else:  
        largest = b  # Statement 3  
    return largest  # Statement 4
```

Total number of executable statements: 4

1. if a > b
2. largest = a
3. largest = b
4. return largest

How to Achieve 100% Statement Coverage?

Formula: Executed statements / Total statements $\times 100$

Test Case 1: $a = 10, b = 5$

Condition $a > b$ is True

Path: Statement 1 $\rightarrow$ Statement 2 $\rightarrow$ Statement 4

Statements Executed: 1, 2, 4

Coverage so far: $3/4 \times 100 = 75\%$

Statement 3 ($largest = b$) has not been executed.

Test Case 2: $a = 5, b = 10$

Condition $a > b$ is False

Path: Statement 1 $\rightarrow$ Statement 3 $\rightarrow$ Statement 4

Statements Executed: 1, 3, 4

Test Case	a	b	Condition	Statements Executed
TC1	10	5	True	1, 2, 4
TC2	5	10	False	1, 3, 4

Total unique statements covered = 1, 2, 3, 4

Statement Coverage = $4/4 \times 100 = 100\%$

```
1. def add_numbers(a, b):  
2.     if a > 0 and b > 0:  
3.         result = a + b  
4.     else:  
5.         result = 0  
6.     return result
```

- To test the statement coverage of this code, write test cases that execute each statement at least once.
- Table shows the minimum set of test cases to achieve 100 percent statement coverage.

Statement Coverage Test Cases (Add Numbers)

Test Case	A	b	Result
1	3	4	7
2	0	2	0

Statement coverage : Example

```
def add_numbers(a,b):  #statement 1
    if a>0 and b>0:    #statement 2
        result = a+b   #statement 3
    else:
        result = 0      #statement 4
    return result       #statement 5
```

Test Case 1: a=2, b=2

Path: Statement 1 → Statement 2 (condition is True) → Statement 3 → Statement 5.

Statements Executed: 1, 2, 3, 5.

Coverage so far: $4/5 = 90\%$

Statement coverage : Example

```
def add_numbers(a,b):  #statement 1
    if a>0 and b>0:    #statement 2
        result = a+b   #statement 3
    else:
        result = 0     #statement 4
    return result      #statement 5
```

Test Case 2: a=0, b=-2

Path: Statement 1 → Statement 2 (condition is False) → Statement 4 → Statement 5.

Statements Executed: 1, 2, 4, 5.

Coverage so far: $4/5 = 90\%$

So totally executed statements :1,2,3,4,5 = 100%

```
def average(numbers):  
    """  
    Returns the average of a list of numbers.  
    """  
    if len(numbers) == 0:  
        return None  
    else:  
        sum = 0  
        for number in numbers:  
            sum += number  
        return sum / len(numbers)
```

- To achieve statement coverage for this function, we would need to execute each line of code at least once during testing.
- This would require at least two test cases: one where the input list is empty, and one where the input list contains at least one number.

Statement Coverage Test Cases (Average)

Test Case	Numbers	Result
1	empty	None
2	1,2,3	2

Statement coverage : Example

```
def average(numbers):           #statement 1
    if len(numbers)==0:         #statement 2
        return None            #statement 3
    else:
        sum = 0                 #statement 4
        for number in numbers:  #statement 5
            sum += number       #statement 6
        return sum/len(numbers) #statement 7
```

Input:

[1,2,3,4,5]:

Output:

sum = 1+2+3+4+5 = 15

len(numbers) = 5

average = 15 / 5 = 3.0

Test Case 1: numbers = [1,2,3,4,5]

Path: Statement 1 → Statement 2 (condition is False) → Statement 4 → Statement 5 → Statement 6 → Statement 7.

Statements Executed: 1, 2, 4, 5, 6, 7.

Coverage so far: 6/7 = 90%

Statement coverage : Example

```
def average(numbers):           #statement 1
    if len(numbers)==0:         #statement 2
        return None            #statement 3
    else:
        sum = 0                 #statement 4
        for number in numbers:  #statement 5
            sum += number       #statement 6
        return sum/len(numbers) #statement 7
```

Input:

[1,2,3,4,5]:

Output:

sum = 1+2+3+4+5 = 15

len(numbers) = 5

average = 15 / 5 = 3.0

Test Case 2: numbers = [0]

Path: Statement 1 → Statement 2 (condition is True) → Statement 3

Statements Executed: 1, 2, 3.

Coverage so far: $3/7 = 40\%$

So totally executed statements :1,2,3,4,5,6,7 = 100%

Conclusion:

- Achieving statement coverage alone is not enough to guarantee that the code is completely tested.
- It is possible for all statements to be executed, but certain logic paths may not be tested.
- Therefore, other testing techniques such as branch coverage and path coverage should also be used in conjunction with statement coverage to ensure thorough testing.

UQ24CA342A Software Testing and Automation

Classroom Activity on Statement coverage Testing

Write statement coverage testing test cases for below code:

```
def classify_number(x):  
    if x > 0:  
        print("Positive")  
    elif x == 0:  
        print("Zero")  
    else:  
        print("Negative")
```

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. What is the statement coverage formula?

- A) Executed statements / Total statements $\times 100$
- B) Total statements / Executed statements $\times 100$
- C) Test cases / Statements $\times 100$
- D) Branches / Statements $\times 100$

2. Which statement is TRUE about Statement Coverage?

- A) Every possible input must be tested
- B) Every executable statement must be executed at least once
- C) Every condition must be True only
- D) Only the if statement needs to be tested

3. Statement Coverage is a type of which testing technique?

- A) Black-box testing
- B) White-box testing
- C) Acceptance testing
- D) Usability testing

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. A
2. B
3. B

UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

Department of Computer Applications

- Path coverage is a white-box testing technique that involves testing all possible paths of a program.
- A path is a sequence of statements that begins with the entry point and ends with the exit point of the program.
- Path coverage aims to test each unique path at least once to ensure that the program is functioning as expected.

- Path coverage is a more thorough testing approach than statement coverage or branch coverage because it takes into account the interaction between different paths in the program.
- path coverage can be time-consuming and difficult to achieve for complex programs.
- it may not be necessary for all programs, depending on the level of risk associated with the software being tested.

```
1. def add_numbers(a, b):  
2.     if a > 0 and b > 0:  
3.         result = a + b  
4.     else:  
5.         result = 0  
6.     return result
```

- To achieve 100 percent path coverage of this code, test all possible paths through the code.
- Table shows the minimum set of test cases to achieve 100 percent path coverage.

Path Coverage (Add Numbers)

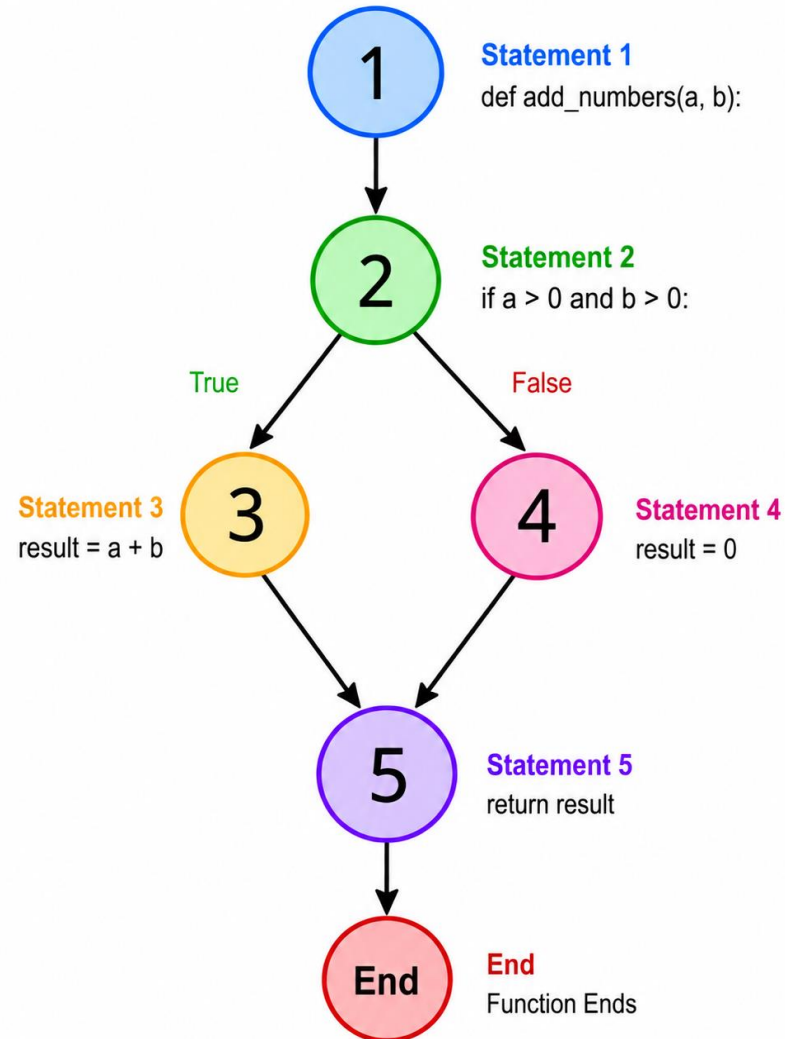
Test Case	A	b	Result
1	3	4	7
2	0	2	0

Path coverage Example

```
def add_numbers(a,b):  #statement 1
    if a>0 and b>0:    #statement 2
        result = a+b   #statement 3
    else:               #statement 4
        result = 0      #statement 5
    return result
```

Path 1: 1,2,3,5

Path 2: 1,2,4,5



Path coverage Example

```
def add_numbers(a,b):  #statement 1
    if a>0 and b>0:    #statement 2
        result = a+b   #statement 3
    else:
        result = 0     #statement 4
    return result      #statement 5
```

Test Case 1: a=2, b=2

Path: Statement 1 → Statement 2 (condition is True)
→ Statement 3

Statements Executed: 1, 2, 3.

Coverage so far: $3/5 = 60\%$

Path coverage Example

```
def add_numbers(a,b):  #statement 1
    if a>0 and b>0:    #statement 2
        result = a+b   #statement 3
    else:
        result = 0      #statement 4
    return result       #statement 5
```

Test Case 2: a=0, b=-2

Path: Statement 1 → Statement 2 (condition is False) → Statement 4 → Statement 5

Statements Executed: 1, 2, 4, 5

Coverage so far: $4/5 = 90\%$

Totally: 1,2,3,4,5 → $5/5 = 100\%$

Path coverage Example

Python function that takes three integers as input and returns the sum of the two largest numbers

```
def sum_of_largest_two(a, b, c):  
    """  
    Returns the sum of the two largest numbers among a, b, and c.  
    """  
    if a >= b and a >= c:  
        if b >= c:  
            return a + b  
        else:  
            return a + c  
    elif b >= a and b >= c:  
        if a >= c:  
            return b + a  
        else:  
            return b + c  
    else:  
        if a >= b:  
            return c + a  
        else:  
            return c + b
```

- To achieve path coverage for this function, execute every possible path through the control flow of the function.
- This would require at least three test cases: one where a is the largest number, one where b is the largest number, and one where c is the largest number.

Path Coverage (Sum of Largest Two)

Test Case	A	b	c	Result
1	1	2	3	5
2	2	1	3	5
3	3	2	1	5

```
def sum_of_largest_two(a,b,c): #statement 1
    if a>=b and a>=c:          #statement 2
        if b >= c:              #statement 3
            return a+b          #statement 4
        else:
            return a+c          #statement 5
    elif b>=a and b>=c:         #statement 6
        if a>=c:               #statement 7
            return b+a          #statement 8
        else:
            return b+c          #statement 9
    else:
        if a>=b:               #statement 10
            return c+a          #statement 11
        else:
            return c+b          #statement 12
```

Path 1: 1,2,3,4

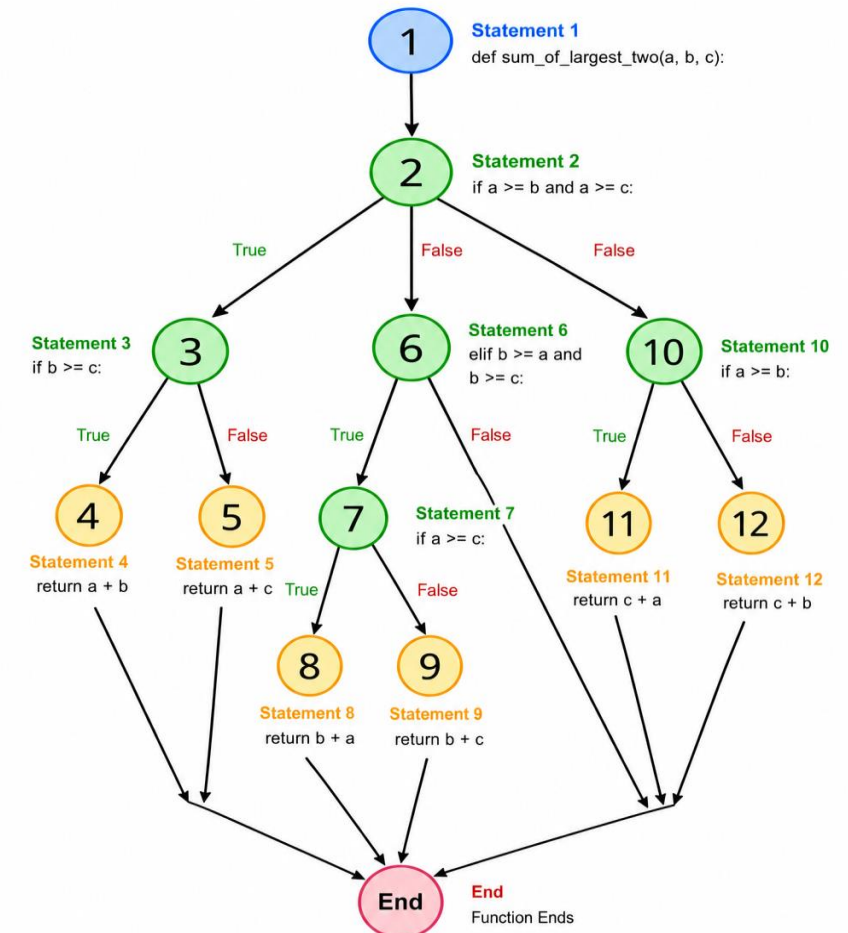
Path 2: 1,2,3,5

Path 3: 1,2,6,7,8

Path 4: 1,2,6,7,9

Path 5: 1,2,10,11

Path 6: 1,2,10,12



UQ24CA342A Software Testing and Automation

Path coverage Example

Test Case 1: $a=4, b=3, c=1$

Statements Executed: 1,2,3,4 ($a+b$)

Test Case 2: $a=4, b=1, c=3$

Statements Executed: 1,2,3,5 ($a+c$)

Test Case 3: $a=3, b=4, c=1$

Statements Executed: 1,2,6,7,8 ($b+a$)

Test Case 4: $a=1, b=4, c=3$

Statements Executed: 1,2,6,7,9 ($b+c$)

Test Case 5: $a=3, b=1, c=4$

Statements Executed: 1,2,10,11 ($c+a$)

Test Case 6: $a=1, b=3, c=4$

Statements Executed: 1,2,10,12 ($c+b$)

UQ24CA342A Software Testing and Automation

Classroom Activity on Path coverage Testing



Write path coverage test cases for below code:

```
def student_result(marks):  
    if marks >= 90:  
        grade = "A"  
    elif marks >= 75:  
        grade = "B"  
    elif marks >= 50:  
        grade = "C"  
    else:  
        grade = "Fail"
```

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. What is the main objective of Path Testing?

- A) To test the user interface
- B) To test different execution paths through a program
- C) To test database performance
- D) To test only input values

2. Which graph is commonly used to represent the flow of control in Path Testing?

- A) Pie chart
- B) Control Flow Graph (CFG)
- C) Bar graph
- D) Gantt chart

3. What does a path in Path Testing represent?

- A) A sequence of statements or nodes executed from entry to exit
- B) A list of software requirements
- C) A sequence of user interface screens only
- D) A list of defects

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. B
2. B
3. A

UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

Department of Computer Applications

- Condition coverage is a type of white-box testing technique that aims to ensure that **all possible combinations of Boolean conditions** in a program have been executed at least once.
- The technique focuses on evaluating the different ways in which a program's conditions can be evaluated as true or false.
- Condition coverage can be applied to both procedural and object-oriented programs, and it is typically used to verify the correctness of complex branching structures, such as loops, if statements, and switch statements.

- Condition coverage requires a test case to cover every possible outcome of each Boolean condition in the program.
- A test case can cover a Boolean condition in one of two ways: (1) it evaluates the condition to true, and (2) it evaluates the condition to false
- In this way, condition coverage ensures that all possible combinations of Boolean conditions are evaluated during the testing process.

Helps identify hidden errors:

- Condition coverage ensures that all possible combinations of Boolean conditions are evaluated during testing.
- This means that even the hidden errors in the program, which might not be evident during normal execution, are discovered.

Increases test case effectiveness:

- Condition coverage ensures that test cases are optimized to find errors.
- By focusing on the different ways in which a program's conditions can be evaluated as true or false, testers can identify and eliminate redundant test cases, thereby increasing the effectiveness of the test cases.

UQ24CA342A Software Testing and Automation

Condition Coverage Disadvantages



Time-consuming:

- Condition coverage requires a large number of test cases, which can be time-consuming to design and execute.

Code complexity:

- Programs with complex branching structures can require a large number of test cases to achieve complete condition coverage.
- This can make it difficult for testers to achieve complete coverage and make it harder to maintain the test suite.

UQ24CA342A Software Testing and Automation

Condition Coverage Example

- To achieve condition coverage, tests would need to be created that cover both the true and false outcomes of both conditions: $a > b$ and $c < d$.
- This would require four tests in total:
 1. $a > b$ is true, $c < d$ is true.
 2. $a > b$ is true, $c < d$ is false.
 3. $a > b$ is false, $c < d$ is true.
 4. $a > b$ is false, $c < d$ is false.

```
if (a > b && c < d) {  
    // do something  
}
```

UQ24CA342A Software Testing and Automation

Condition Coverage Example

- Python function that classifies a triangle based on the lengths of its sides

```
def classify_triangle(a, b, c):  
    """  
    Classifies a triangle with sides of length a, b, and c  
    """  
    if a <= 0 or b <= 0 or c <= 0:  
        return "Invalid"  
    if a == b and b == c:  
        return "Equilateral"  
    if a == b or b == c or c == a:  
        return "Isosceles"  
    return "Scalene"
```

- To achieve condition coverage, tests would need to be created that cover both the true and false outcomes of all conditions:
- $a \leq 0$, $b \leq 0$, and $c \leq 0$, $a == b$, $b == c$, $c == a$

UQ24CA342A Software Testing and Automation

Condition Coverage Example

This would require two tests in total:

1. $a \leq 0$ is true and $b \leq 0$ is true and $c \leq 0$ is true and
 $a == b$ is false and $b == c$ is false and $c == a$ is false.
2. $a \leq 0$ is false and $b \leq 0$ is false and $c \leq 0$ is false and
 $a == b$ is true and $b == c$ is true and $c == a$ is true.

UQ24CA342A Software Testing and Automation

Condition Coverage Example

```
def classify_triangle(a, b, c):  
  
    if a <= 0 or b <= 0 or c <= 0:  
        return "Invalid"  
  
    if a == b and b == c:  
        return "Equilateral"  
  
    if a == b or b == c or c == a:  
        return "Isosceles"  
  
    return "Scalene"
```

UQ24CA342A Software Testing and Automation

Condition Coverage Example

```
def classify_triangle(a, b, c):           #statement 1

    if a <= 0 or b <= 0 or c <= 0:       #statement 2
        return "Invalid"                 #statement 3

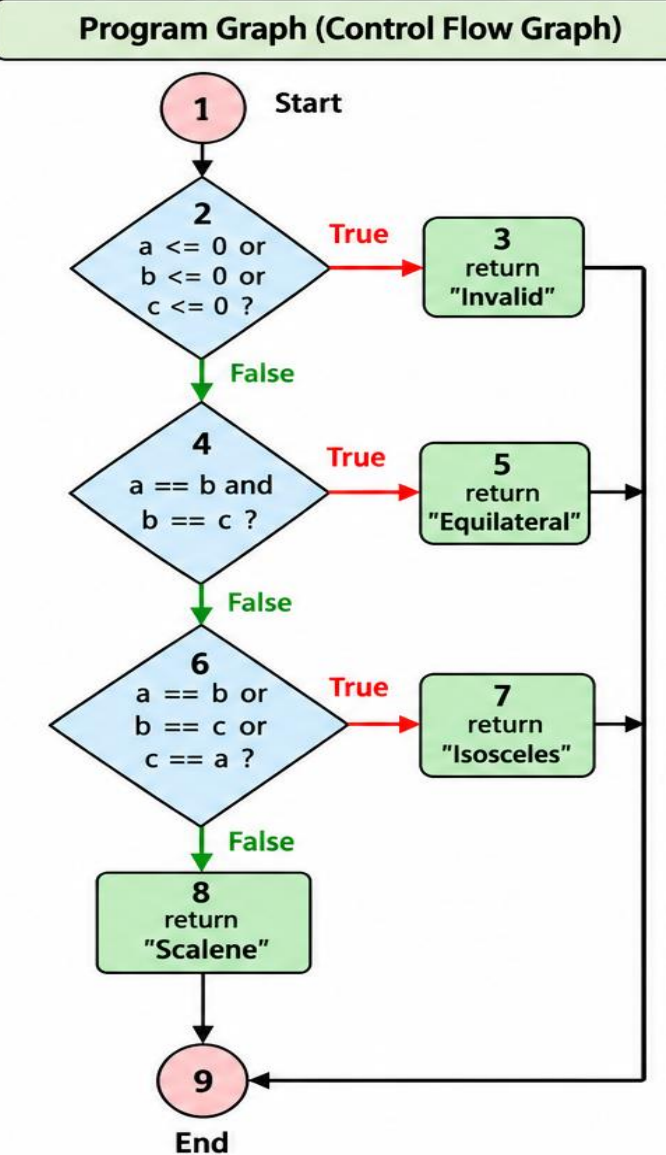
    if a == b and b == c:                 #statement 4
        return "Equilateral"             #statement 5

    if a == b or b == c or c == a:        #statement 6
        return "Isosceles"               #statement 7

    return "Scalene"                       #statement 8
```

UQ24CA342A Software Testing and Automation

Condition Coverage Example



Independent Paths

P1 : 1 → 2 → 3 → 9 (Invalid)
P2 : 1 → 2 → 4 → 5 → 9 (Equilateral)
P3 : 1 → 2 → 4 → 6 → 7 → 9 (Isosceles)
P4 : 1 → 2 → 4 → 6 → 8 → 9 (Scalene)

UQ24CA342A Software Testing and Automation

Condition Coverage Example

ID	Condition
C1	$a \leq 0$
C2	$b \leq 0$
C3	$c \leq 0$
C4	$a == b$ in Equilateral
C5	$b == c$ in Equilateral
C6	$a == b$ in Isosceles
C7	$b == c$ in Isosceles
C8	$c == a$ in Isosceles

UQ24CA342A Software Testing and Automation

Condition Coverage Example

TC	Input (a,b,c)	Result
TC1	(-1, 2, 3)	Invalid
TC2	(1, -2, 3)	Invalid
TC3	(1, 2, -3)	Invalid
TC4	(5, 5, 5)	Equilateral
TC5	(5, 5, 6)	Isosceles
TC6	(5, 6, 6)	Isosceles
TC7	(5, 6, 7)	Scalene

UQ24CA342A Software Testing and Automation

Classroom Activity on Condition Coverage Testing



```
def login(username, password, otp):  
    if username_valid and password_valid and otp_valid:  
        return "Login Successful"  
    else:  
        return "Login Failed"
```

- 1) Prepare test cases so that each condition becomes True at least once and False at least once.
- 2) Determine whether your test cases achieve 100% Condition Coverage.

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. What is the main objective of Condition Coverage Testing?

- A) To test every statement
- B) To test every possible input
- C) To ensure each condition evaluates to both True and False
- D) To test only error conditions

2. Consider the condition:

if A and B:

For 100% Condition Coverage, which is required?

- A) A and B must always be True
- B) A and B must each evaluate to True and False at least once
- C) The complete condition must be True and False only once
- D) Only A needs to be tested

3. Which statement is TRUE about Condition Coverage?

- A) It focuses on individual Boolean conditions
- B) It tests only output values
- C) It does not require Boolean expressions
- D) It guarantees 100% Path Coverage

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. C
2. B
3. A

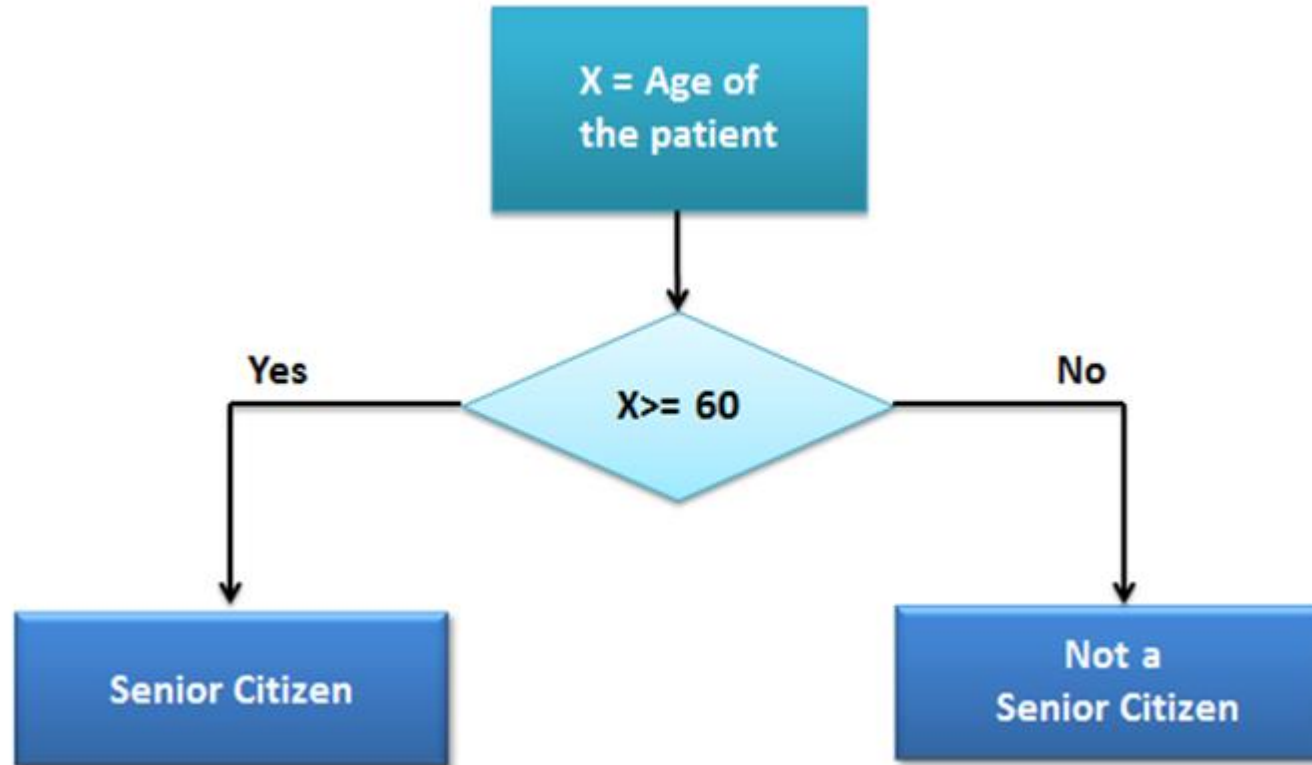
UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

Department of Computer Applications

- Decision coverage is a white-box testing technique that is used to measure the effectiveness of testing by checking if all decisions made in the source code have been exercised.
- A decision is a condition that can lead to either true or false outcome. Decision coverage measures the percentage of decisions that have been executed during testing.
- The goal of decision coverage is to ensure that each decision in the code has been executed at least once and that all possible outcomes (true and false) have been tested.
- To achieve decision coverage, testers must create test cases that execute every decision point in the code.



- In this example, the variable X is assigned to the age of the patient.
- The decision box here is used to evaluate if the age is greater than or equal to the number '60'. If the age is less than provided value, that is 60, then the patient is not labeled as a senior citizen.
- And, if the condition is met and found to be true, then the patient can be marked as a senior citizen.
- The process of performing this evaluation in terms of the modular functionality, without any leakage, can be defined as the practice of the Decision Coverage validation.

```
if (a > b && c < d) {  
    // do something  
}
```

- To achieve decision coverage, we need to ensure that both possible outcomes of the if statement have been executed.
- This would require two tests in total.
 1. $a > b$ is true; $c < d$ is true.
 2. $a > b$ is false; $c < d$ is false.

- Python function that classifies a triangle based on the lengths of its sides:

```
def classify_triangle(a, b, c):  
    """  
    Classifies a triangle with sides of length a, b, and c  
    """  
    if a <= 0 or b <= 0 or c <= 0:  
        return "Invalid"  
    if a == b and b == c:  
        return "Equilateral"  
    if a == b or b == c or c == a:  
        return "Isosceles"  
    return "Scalene"
```

UQ24CA342A Software Testing and Automation

Decision Coverage Example

```
def classify_triangle(a, b, c):  
  
    if a <= 0 or b <= 0 or c <= 0:  
        return "Invalid"  
  
    if a == b and b == c:  
        return "Equilateral"  
  
    if a == b or b == c or c == a:  
        return "Isosceles"  
  
    return "Scalene"
```

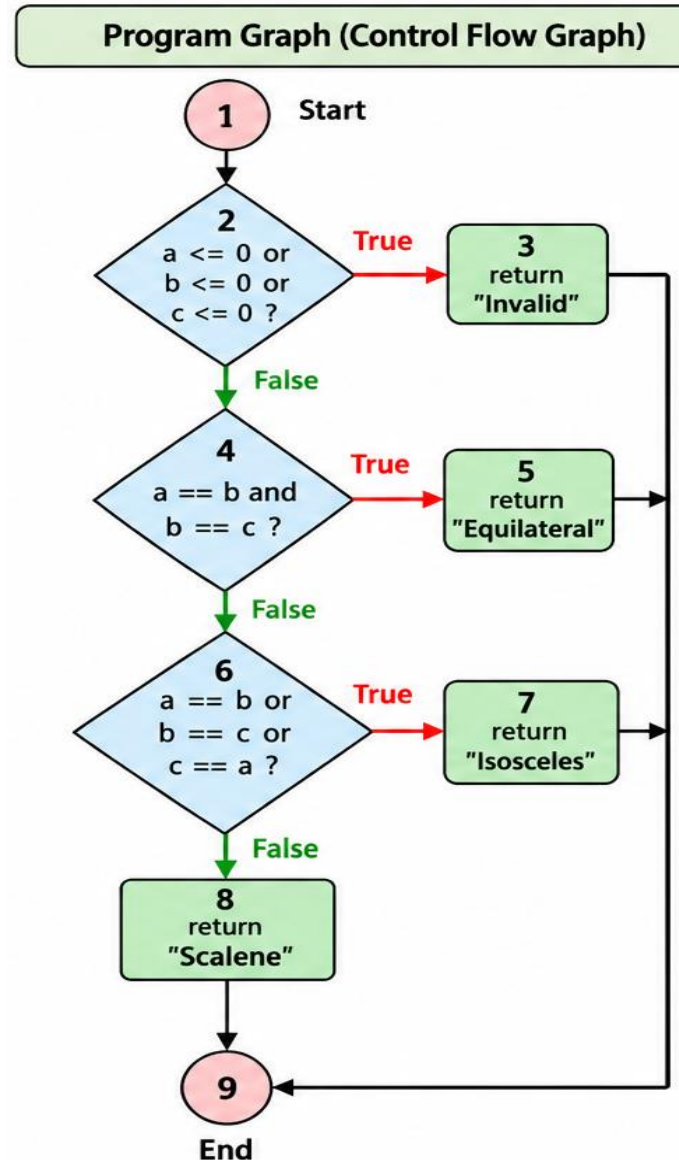
```
def classify_triangle(a, b, c):           #statement 1

    if a <= 0 or b <= 0 or c <= 0:       #statement 2
        return "Invalid"                 #statement 3

    if a == b and b == c:                 #statement 4
        return "Equilateral"             #statement 5

    if a == b or b == c or c == a:        #statement 6
        return "Isosceles"               #statement 7

    return "Scalene"                       #statement 8
```



Independent Paths

P1 : 1 → 2 → 3 → 9 (Invalid)

P2 : 1 → 2 → 4 → 5 → 9 (Equilateral)

P3 : 1 → 2 → 4 → 6 → 7 → 9 (Isosceles)

P4 : 1 → 2 → 4 → 6 → 8 → 9 (Scalene)

UQ24CA342A Software Testing and Automation

Decision Coverage

Decision	Decision
D1	$a \leq 0$ or $b \leq 0$ or $c \leq 0$
D2	$a == b$ and $b == c$
D3	$a == b$ or $b == c$ or $c == a$

Required Test Cases are:

1.D1 is True

2.D1 is False

1.D2 is True

2.D2 is False

1.D3 is True

2.D3 is False

- To achieve decision coverage, we need to ensure that both possible outcomes of the if statements have been executed.
- This would require six tests in total.
 1. If $a \leq 0$ or $b \leq 0$ or $c \leq 0$ is true
 2. If $a \leq 0$ or $b \leq 0$ or $c \leq 0$ is false
 3. If $a == b$ and $b == c$ is true
 4. If $a == b$ and $b == c$ is false
 5. If $a == b$ or $b == c$ or $c == a$ is true
 6. If $a == b$ or $b == c$ or $c == a$ is false

- To achieve decision coverage, we need to ensure that both possible outcomes of the if statements have been executed.
- This would require six tests in total.
 1. If $a \leq 0$ or $b \leq 0$ or $c \leq 0$ is true
 2. If $a \leq 0$ or $b \leq 0$ or $c \leq 0$ is false
 3. If $a == b$ and $b == c$ is true
 4. If $a == b$ and $b == c$ is false
 5. If $a == b$ or $b == c$ or $c == a$ is true
 6. If $a == b$ or $b == c$ or $c == a$ is false

This would require two tests in total:

1. $a \leq 0$ is true and $b \leq 0$ is true and $c \leq 0$ is true and $a == b$ is false and $b == c$ is false and $c == a$ is false.
2. $a \leq 0$ is false and $b \leq 0$ is false and $c \leq 0$ is false and $a == b$ is true and $b == c$ is true and $c == a$ is true.

Test Case	Input (a,b,c)	D1: $a \leq 0$ or $b \leq 0$ or $c \leq 0$	D2: $a == b$ and $b == c$	D3: $a == b$ or $b == c$ or $c == a$	Output
TC1	(0, 5, 6)	True	False	False	Invalid
TC2	(5, 5, 5)	False	True	True	Equilateral
TC3	(5, 5, 6)	False	False	True	Isosceles
TC4	(5, 6, 7)	False	False	False	Scalene

Condition Coverage	Decision Coverage
Focuses on individual conditions within a decision.	Focuses on the overall decision outcome.
Each individual condition must be True and False at least once.	Each decision must produce True and False outcomes at least once.
Used when a decision contains multiple conditions.	Can be applied to any decision such as if, if-else, etc.
Checks the behavior of each Boolean condition.	Checks whether each branch/outcome of the decision is executed.
Example: A and B → both A and B should be True and False.	Example: A and B → the overall expression should be True and False.
May require test cases that do not necessarily execute every branch.	100% Decision Coverage requires execution of every decision outcome.
Condition Coverage → “Did each condition become True and False?” Decision Coverage → “Did the overall decision take both True and False branches?”	

UQ24CA342A Software Testing and Automation

Classroom Activity on Decision Coverage Testing



```
def calculate_discount(amount):
```

```
    if amount >= 1000:
```

```
        discount = 10
```

```
    else:
```

```
        discount = 0
```

```
    return discount
```

1. Design test cases that execute both outcomes of the decision.
2. Calculate the Decision Coverage.

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. What is the main objective of Decision Coverage Testing?

- A) To test every statement
- B) To test every possible input
- C) To ensure every decision outcome is executed
- D) To test only invalid inputs

2. Which of the following is the correct formula for Decision Coverage?

- A) Executed statements / Total statements $\times 100$
- B) Executed decision outcomes / Total decision outcomes $\times 100$
- C) Test cases / Total decisions $\times 100$
- D) Executed conditions / Total conditions $\times 100$

3. A program has 4 decision outcomes. During testing, 3 outcomes are executed. What is the Decision Coverage?

- A) 25%
- B) 50%
- C) 75%
- D) 100%

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. C
2. B
3. C

UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

Department of Computer Applications

- Multiple condition coverage (MCC) is a white-box testing technique used to ensure that all possible combinations of Boolean conditions are tested at least once.
- MCC is a stronger form of condition coverage.
- To achieve MCC, all possible combinations of each condition in the decision must be evaluated.
- For example, if a decision has two conditions, A and B, then four test cases are needed to cover all possible combinations (true/true, true/false, false/true, and false/false).

UQ24CA342A Software Testing and Automation

Multiple Condition Coverage



- MCC is a powerful technique, as it ensures that all combinations of conditions are tested, which can help uncover complex logic errors.
- However, it can also result in a large number of test cases, which can be difficult to manage.
- Therefore, it is important to use MCC judiciously and to combine it with other testing techniques, such as decision coverage, to achieve a more comprehensive test suite.

```
function getResult(x, y, z) {  
    if (x > 0 && y > 0 && z > 0) {  
        return "Positive";  
    } else if (x < 0 && y < 0 && z < 0) {  
        return "Negative";  
    } else {  
        return "Mixed";  
    }  
}
```

- To achieve multiple condition coverage for this function, test all possible combinations of truth and falsehood for each condition.
- There are three conditions, each with two possible outcomes, so there are a total of $2^3 = 8$ possible combinations to test.

UQ24CA342A Software Testing and Automation

Multiple Condition Coverage Example

```
def get_result(x, y, z):  
    if x > 0 and y > 0 and z > 0:  
        return "Positive"  
    elif x < 0 and y < 0 and z < 0:  
        return "Negative"  
    else:  
        return "Mixed"
```

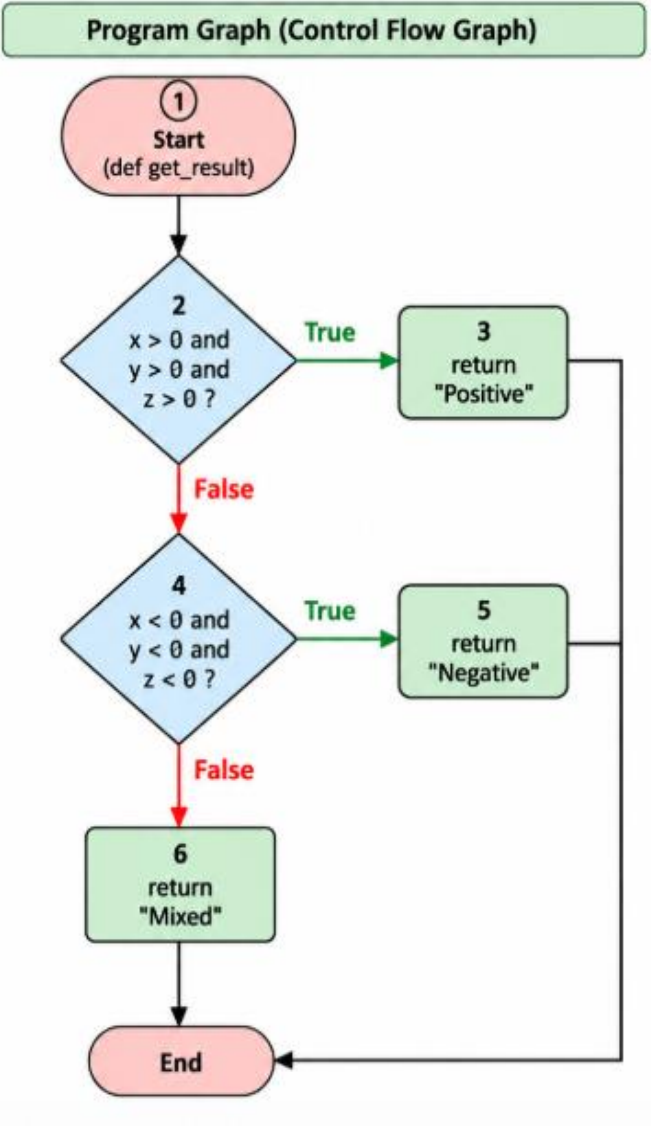
UQ24CA342A Software Testing and Automation

Multiple Condition Coverage Example

def get_result(x, y, z):	#statement 1
if x > 0 and y > 0 and z > 0:	#statement 2
return "Positive"	#statement 3
elif x < 0 and y < 0 and z < 0:	#statement 4
return "Negative"	#statement 5
else:	
return "Mixed"	#statement 6

UQ24CA342A Software Testing and Automation

Multiple Condition Coverage Example



Condition	Meaning
C1	$x > 0$
C2	$y > 0$
C3	$z > 0$
C4	$x < 0$
C5	$y < 0$
C6	$z < 0$

UQ24CA342A Software Testing and Automation

Multiple Condition Coverage Example

We can use a truth table to keep track of which combinations we have tested.

By testing all eight combinations, we can be sure that we have achieved multiple condition coverage for this function.

MCC (getResult)

x>0	y>0	z>0	Expected Result
T	T	T	Positive
T	T	F	Mixed
T	F	T	Mixed
T	F	F	Mixed
F	T	T	Mixed
F	T	F	Mixed
F	F	T	Mixed
F	F	F	Negative

UQ24CA342A Software Testing and Automation

Multiple Condition Coverage Example

Consider this function that calculates the shipping cost for a package based on its weight, size, and destination:

```
def calculate_shipping(weight, size, destination):  
    """  
    Calculates the shipping cost for a package based on  
    """  
    if weight <= 0 or weight > 50:  
        return "Invalid weight"  
    elif size <= 0 or size > 100:  
        return "Invalid size"  
    elif destination not in ["USA", "Canada", "Mexico"]:  
        return "Invalid destination"
```

```
elif weight <= 10:  
    if size <= 20:  
        return 10  
    elif size <= 50:  
        return 20  
    else:  
        return 30  
else:  
    if size <= 20:  
        return 20  
    elif size <= 50:  
        return 30  
    else:  
        return 40
```

UQ24CA342A Software Testing and Automation

Multiple Condition Coverage Example

```
def calculate_shipping(weight, size, destination):  
    if weight <= 0 or weight > 50:  
        return "Invalid weight"  
    elif size <= 0 or size > 100:  
        return "Invalid size"  
    elif destination not in ["USA", "Canada", "Mexico"]:  
        return "Invalid destination"
```

```
else:  
    if weight <= 10:  
        if size <= 20:  
            return 10  
        elif size <= 50:  
            return 20  
        else:  
            return 30
```

UQ24CA342A Software Testing and Automation

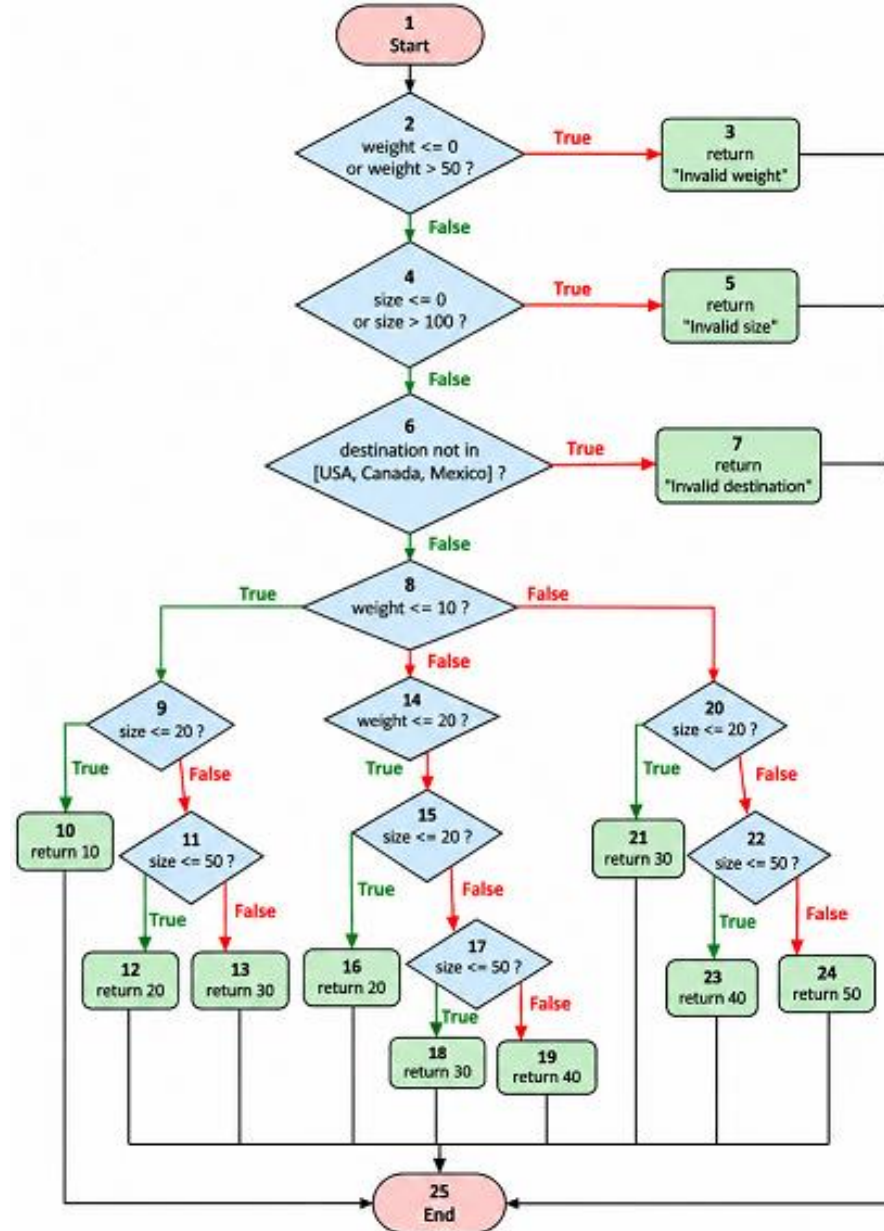
Multiple Condition Coverage Example

```
elif weight <= 20:  
    if size <= 20:  
        return 20  
    elif size <= 50:  
        return 30  
    else:  
        return 40
```

```
else:  
    if size <= 20:  
        return 30  
    elif size <= 50:  
        return 40  
    else:  
        return 50
```

UQ24CA342A Software Testing and Automation

Multiple Condition Coverage Example



UQ24CA342A Software Testing and Automation

Multiple Condition Coverage Example

There are multiple Boolean conditions that need to be evaluated for multiple condition coverage:

- `weight <= 0 or weight > 50`
- `size <= 0 or size > 100`
- `destination not in ["USA", "Canada", "Mexico"]`
- `weight <= 10`
- `size <= 20`
- `size <= 50`

UQ24CA342A Software Testing and Automation

Multiple Condition Coverage Example



Table shows a possible set of tests.

Table *MCC (calculate_shipping)*

weight	size	destination	Expected Result
5	10	“USA”	10
5	30	“USA”	20
15	70	“Canada”	40
-5	10	“Mexico”	“Invalid weight”
5	-10	“USA”	“Invalid size”
5	10	“Japan”	“Invalid destination”

```
def check_prime_offer(age, prime_member, order_amount):  
    if age >= 18 and prime_member == True and order_amount >= 499:  
        return "Prime Offer Available"  
    else:  
        return "Prime Offer Not Available"
```

Design MCC Test Cases Table

1. What is the main objective of Multiple Condition Coverage (MCC)?

- A) To execute every statement once
- B) To test every branch once
- C) To test every possible combination of conditions
- D) To test only valid inputs

2. If a decision contains 3 independent conditions, how many possible combinations must be tested for 100% MCC?

- A) 3
- B) 6
- C) 8
- D) 9

3. What is the formula for the number of possible combinations in MCC?

- A) $n \times 2$
- B) 2^n
- C) n^2
- D) $n + 2$

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. C
2. C
3. B

UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

Department of Computer Applications

- MC/DC is a white-box testing technique that ensures all possible combinations of conditions and decisions have been executed in a software system.
- It is considered to be one of the most stringent testing techniques and is often used in safety-critical systems such as aviation, defense, and medical devices.
- The MC/DC technique requires each condition to be evaluated as true and false at least once and that each decision is tested under all possible combinations of conditions.

- The MC/DC technique can be applied at various stages of the software development life cycle, including the requirements phase, design phase, and testing phase.
- It can also be used in conjunction with other testing techniques to ensure complete code coverage.
- However, implementing MC/DC can be time-consuming and requires a high level of expertise, as it involves understanding the code logic and identifying all possible decision combinations.

UQ24CA342A Software Testing and Automation

Modified Condition/Decision Coverage Example

```
function checkValues(a, b, c) {  
    if (a > 0 && b > 0 || c > 0) {  
        return true;  
    } else {  
        return false;  
    }  
}
```

- First we create a truth table.
- We can use the truth table shown in Table to keep track of all possible combinations.

UQ24CA342A Software Testing and Automation

Modified Condition/Decision Coverage Example

- First we create a truth table.
- We can use the truth table shown in Table to keep track of all possible combinations.

Table Truth Table (checkValues)

Test Case	a>0	b>0	c>0	Expected Result
1	T	T	T	true
2	T	T	F	true
3	T	F	T	true
4	T	F	F	false
5	F	T	T	true
6	F	T	F	false
7	F	F	T	true
8	F	F	F	false

UQ24CA342A Software Testing and Automation

Modified Condition/Decision Coverage Example

- Now find the test cases where changing a condition's input affects the whole decision.
- 1. for the **condition $a > 0$** to be the decisive condition, use **test cases 2 and 6**.
- 2. for the **condition $b > 0$** to be the decisive condition, use **test cases 2 and 4**.
- 3. for the **condition $c > 0$** to be the decisive condition, use **test cases 3 and 4**
(5 and 6 would also be correct).
- This leads to the set of test cases shown in Table .

UQ24CA342A Software Testing and Automation

Modified Condition/Decision Coverage Example



Table *MCDC (checkValues)*

Test Case	a>0	b>0	c>0	Expected Result
1	T	T	F	true
2	T	F	T	true
3	T	F	F	false
4	F	T	F	false

UQ24CA342A Software Testing and Automation

Modified Condition/Decision Coverage

```
def check_eligibility(attendance, marks, fees_paid):  
    if attendance >= 75 and marks >= 40 and fees_paid == True:  
        return "Eligible"  
    else:  
        return "Not Eligible"
```

UQ24CA342A Software Testing and Automation

Modified Condition/Decision Coverage

```
def check_eligibility(attendance, marks, fees_paid):  
    if attendance >= 75 and marks >= 40 and fees_paid == True:  
        return "Eligible"  
    else:  
        return "Not Eligible"
```

Condition:

C1: attendance >= 75

C2: marks >= 40

C3: fees_paid == True

UQ24CA342A Software Testing and Automation

Modified Condition/Decision Coverage

MC/DC Test Cases

Test Case	Attendance	Marks	Fees Paid	C1	C2	C3	Decision	Result
TC1	80	50	True	T	T	T	T	Eligible
TC2	70	50	True	F	T	T	F	Not Eligible
TC3	80	30	True	T	F	T	F	Not Eligible
TC4	80	50	False	T	T	F	F	Not Eligible

UQ24CA342A Software Testing and Automation

Modified Condition/Decision Coverage

How MC/DC is demonstrated

Condition	Compare	What changes?
C1	TC1 $\rightarrow$ TC2	C1: T $\rightarrow$ F, decision T $\rightarrow$ F
C2	TC1 $\rightarrow$ TC3	C2: T $\rightarrow$ F, decision T $\rightarrow$ F
C3	TC1 $\rightarrow$ TC4	C3: T $\rightarrow$ F, decision T $\rightarrow$ F

In each pair, only one condition changes, while the other two remain the same.

Therefore:

4 test cases $\rightarrow$ 100% MC/DC coverage.

UQ24CA342A Software Testing and Automation

Modified Condition/Decision Coverage

Conclusion:

- The idea behind MCDC is similar to the algorithm of the Karnaugh map. MCDC's benefit is that you can test all the possible combinations of conditions and decisions with fewer test cases
- It is encouraged to be used in situations where we don't have enough time or resources to execute the complete set of combinations.
- For a given N number of inputs, you always get N+1 test cases.
- In our example, we had three inputs ($a > 0$, $b > 0$, $c > 0$) and the test cases are four (instead of eight for MCC).
- If we had four inputs ($a > 0$, $b > 0$, $c > 0$, $d > 0$), the test cases would be five (instead of sixteen for MCC).

UQ24CA342A Software Testing and Automation

Classroom Activity on MC/DC Testing

```
def check_order(amount, prime_member, in_stock):  
    if amount >= 1000 and prime_member == True and in_stock == True:  
        return "Order Approved"  
    else:  
        return "Order Rejected"
```

1. Identify the three individual conditions.
2. Prepare test cases for the decision.
3. For each condition, select two test cases where only that condition changes.
4. Check whether changing that condition changes the overall decision outcome.
5. Determine whether the test set achieves 100% MC/DC Coverage.

1. What is the main objective of MC/DC testing?

- A) To execute every statement at least once
- B) To test every possible input combination
- C) To show that each condition can independently affect the overall decision outcome
- D) To test only the True branch

3. Which statement correctly describes MC/DC testing?

- A) It requires testing every possible combination of conditions
- B) It requires every statement to execute at least twice
- C) It demonstrates that each individual condition independently influences the decision outcome
- D) It tests only the overall decision as True and False

UQ24CA342A Software Testing and Automation

Classroom Quiz

2. if A and B:

 result = "Yes"

else:

 result = "No"

Which pair of test cases can demonstrate that **A independently affects** the decision?

A) A=True, B=True → Yes

 A=False, B=True → No

B) A=True, B=True → Yes

 A=True, B=False → No

C) A=False, B=True → No

 A=False, B=False → No

D) A=True, B=False → No

 A=False, B=False → No

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. C
2. A
3. C

UQ24CA342A Software Testing and Automation

Test Design Techniques

Mrs. Sowmya BP

Department of Computer Applications

- Loop testing is a technique used in software testing to ensure that loops within a program are functioning correctly.
- This type of testing is used to identify defects or errors that can occur when a program is executed multiple times, as is often the case with loops.
- The goal of loop testing is to ensure that the loop is executed the correct number of times, that the loop is exited when it is supposed to be exited, and that the loop is executed with the correct inputs and outputs.
- This is accomplished by testing the loop under a variety of conditions, including boundary conditions and error conditions.

Different types of loop testing:

1. Simple loop testing
2. Nested loop testing
3. Concatenated loop testing
4. Iterative loop testing

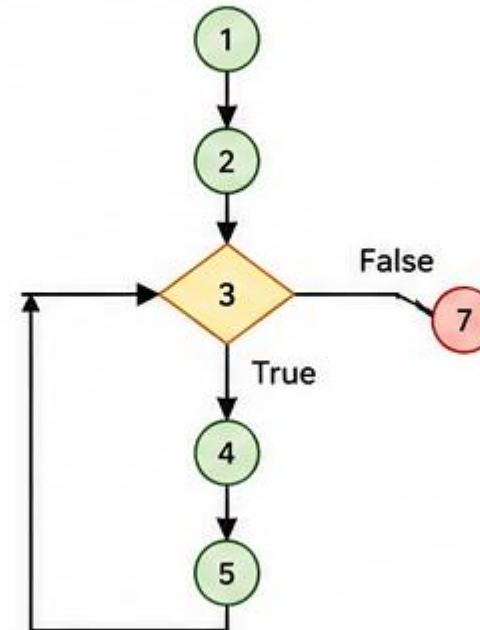
Simple loop testing: In this technique, the loop is executed with the minimum and maximum values of the loop index, as well as with values that are in between the minimum and maximum values. This is done to ensure that the loop is executing the correct number of times.

1. SIMPLE LOOP TESTING

Program Example (Pseudo Code)

```
1 sum = 0
2 i = 1
3 while (i <= 5) {
4     sum = sum + i
5     i = i + 1
6 }
7 print(sum)
```

Control Flow Graph



Test Cases

- 0 iteration (skip loop)
Path: 1-2-3-7
- 1 iteration
Path: 1-2-3-4-5-3-7
- 2 iterations
Path: 1-2-3-4-5-3-4-5-3-7
- n iterations
Path: 1-2-(3-4-5)ⁿ-3-7

Nested loop testing: In this technique, the loops that are nested within the main loop are tested individually, as well as together. This is done to ensure that the loops are interacting correctly with one another.

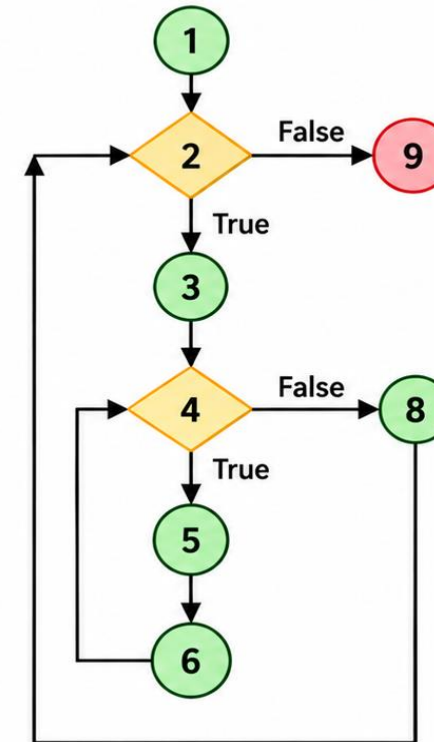
2. NESTED LOOP TESTING

Program Example (Pseudo Code)

```

1  i = 1
2  while (i <= 2) {
3      j = 1
4      while (j <= 3) {
5          print(i, j)
6          j = j + 1
7      }
8      i = i + 1
9  End
    
```

Control Flow Graph



Test Cases

Test Case	Outer Loop (i <= 2)	Inner Loop (j <= 3)	Path
TC1	0 times (i = 3)	0 times	1-2-9
TC2	1 time (i = 1)	0 times (j = 4)	1-2-3-4-8-2-9
TC3	1 time (i = 1)	1 time (j = 1)	1-2-3-4-5-6-4-8-2-9
TC4	2 times (i = 1, 2)	3 times (j = 1, 2, 3) (normal)	1-2-3-4-5-6-4-5-6-4-8-2-9

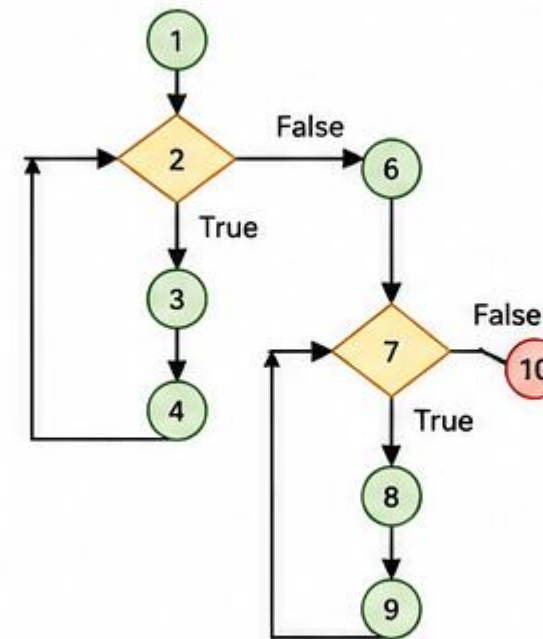
Concatenated loop testing: In this technique, multiple loops are executed together to test their interaction. This is done to ensure that the loops are executing in the correct order and that they are interacting correctly with one another.

3. CONCATENATED LOOP TESTING

Program Example (Pseudo Code)

```
1 i = 1
2 while (i <= 2) {
3   print("A", i)
4   i = i + 1
5 }
6 j = 1
7 while (j <= 3) {
8   print("B", j)
9   j = j + 1
10 }
```

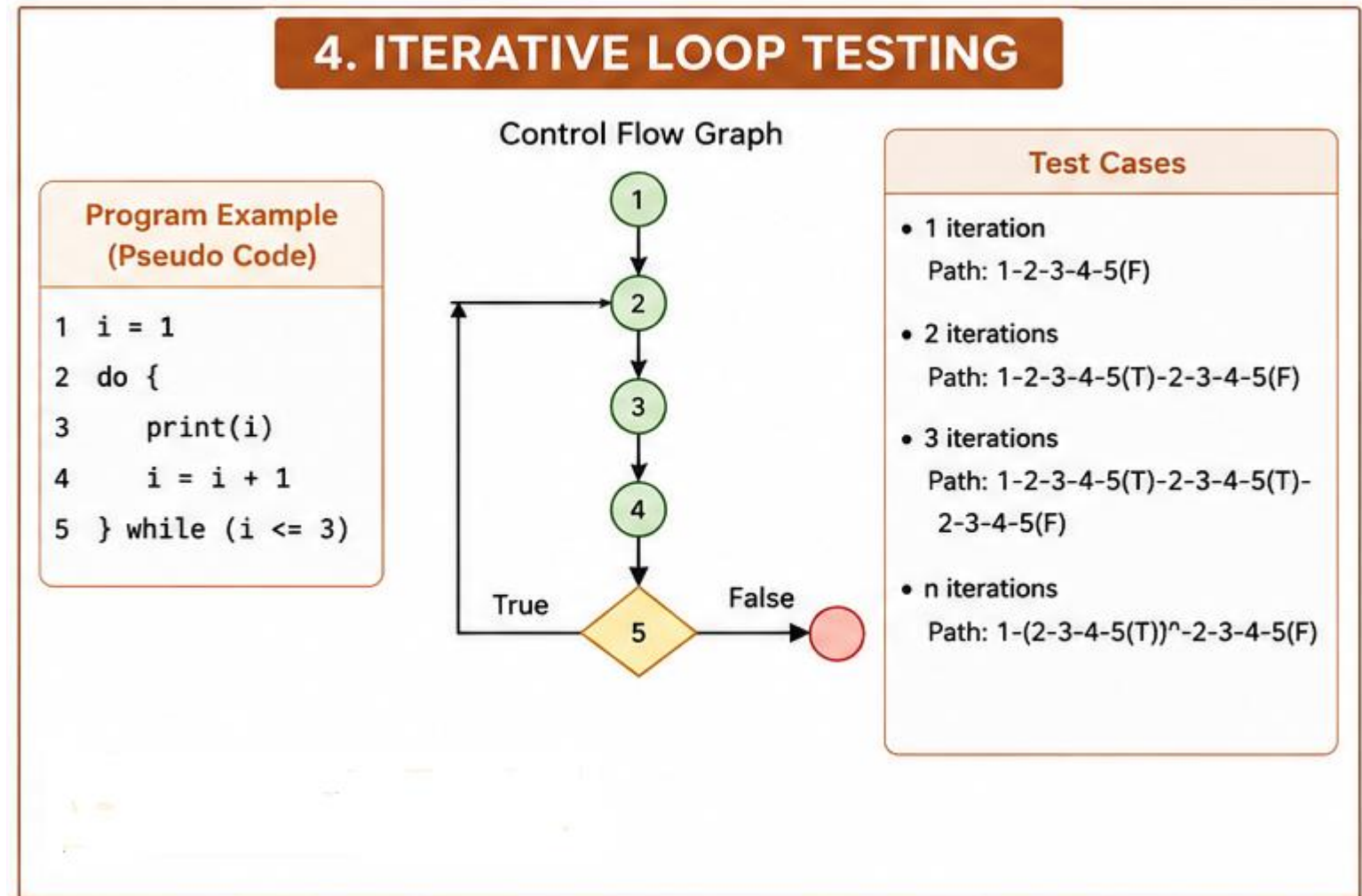
Control Flow Graph



Test Cases

- First loop 0 time, second loop 0 time
Path: 1-2-6-7-10
- First 1 time, second 0 time
Path: 1-2-3-4-2-6-7-10
- First 0 time, second 1 time
Path: 1-2-6-7-8-9-7-10
- First 2 times, second 3 times
Path: 1-2-(3-4-2)²-6-7-(8-9-7)³-10

Iterative loop testing: In this technique, the loop is executed with a variety of input values to ensure that the loop is functioning correctly under different conditions.



Test Cases

- 1 iteration
Path: 1-2-3-4-5(F)
- 2 iterations
Path: 1-2-3-4-5(T)-2-3-4-5(F)
- 3 iterations
Path: 1-2-3-4-5(T)-2-3-4-5(T)-2-3-4-5(F)
- n iterations
Path: 1-(2-3-4-5(T))^n-2-3-4-5(F)

```
function sumArray(arr) {  
  let sum = 0;  
  for (let i = 0; i < arr.length; i++) {  
    sum += arr[i];  
  }  
  return sum;  
}
```

This function takes an array of numbers and returns the sum of all the numbers in the array.

The for loop iterates over each element in the array and adds it to the sum variable.

To test this function using loop testing, we would want to design our test cases to exercise the loop as thoroughly as possible. The following are some examples of test cases can consider:

- An array with no elements
- An array with one element
- An array with two elements
- An array with many elements
- An array with all elements negative
- An array with all elements positive
- An array with a mix of positive and negative elements
- An array with large numbers that could cause overflow

Conclusion:

- Test function with a variety of inputs to ensure that it handles edge cases and potential issues with the loop, such as off-by-one errors, infinite loops, and so on.
- Loop testing can be time-consuming, as it requires running the loop multiple times with different inputs. However, it can be an effective way to identify and correct issues related to loops in code.

UQ24CA342A Software Testing and Automation

Classroom Activity on Loop Testing

```
total = 0
```

```
i = 1
```

```
while i <= 5:
```

```
    mark = int(input("Enter mark: "))
```

```
    total = total + mark
```

```
    i = i + 1
```

```
average = total / 5
```

```
print(average)
```

	Loop Type	Student Activity
1	Simple Loop Testing	Identify test cases for 0, 1, 2, 5, and more than 5 iterations.
2	Nested Loop Testing	Modify the program to process 3 students × 2 subjects and identify test cases for both loops.
3	Concatenated Loop Testing	Create two loops: one for reading marks and another for displaying marks. Identify test cases for both loops.
4	Iterative Loop Testing	Test the loop repeatedly with different numbers of students: 1, 2, 5, 10. Identify whether the loop terminates correctly.

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. Which type of loop contains one loop inside another loop?

- A. Simple loop
- B. Concatenated loop
- C. Nested loop
- D. Iterative loop

2. What is the main purpose of loop testing?

- A. To test the user interface
- B. To test the execution of loops for different iterations
- C. To test database connections
- D. To test network security

3. In concatenated loop testing, loops are:

- A. Placed one inside another
- B. Executed one after another
- C. Always executed only once
- D. Never executed independently

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. C
2. B
3. B

UQ24CA342A Software Testing and Automation

Test Design Techniques

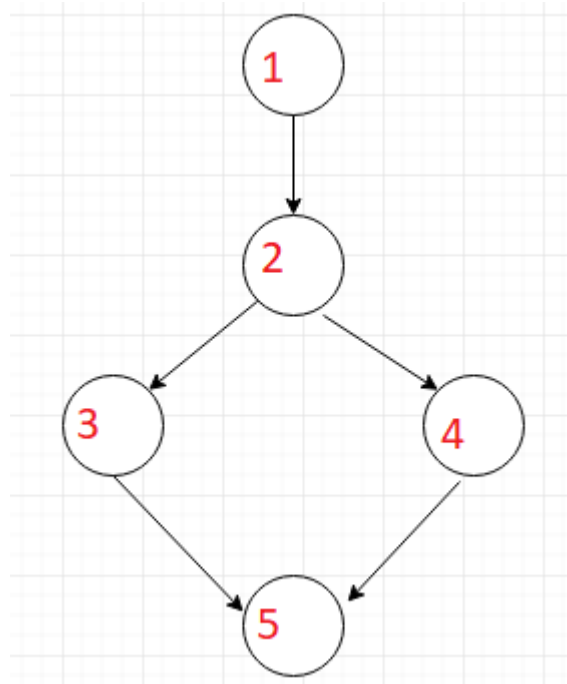
Mrs. Sowmya BP

Department of Computer Applications

- Data flow testing is a technique that focuses on the flow of data within the software being tested.
- It helps to identify whether the data is being processed correctly as it moves through the different components of the software.
- This testing technique can identify vulnerabilities, such as buffer overflows, and ensure that the software is free of defects that may occur because of the improper flow of data.

- Example

1. read x, y;
2. if(x>y)
3. a = x+1
- else
4. a = y-1
5. print a;



Define/use of variables of above example:

VARIABLE	DEFINED AT NODE	USED AT NODE
x	1	2, 3
y	1	2, 4
a	3, 4	5

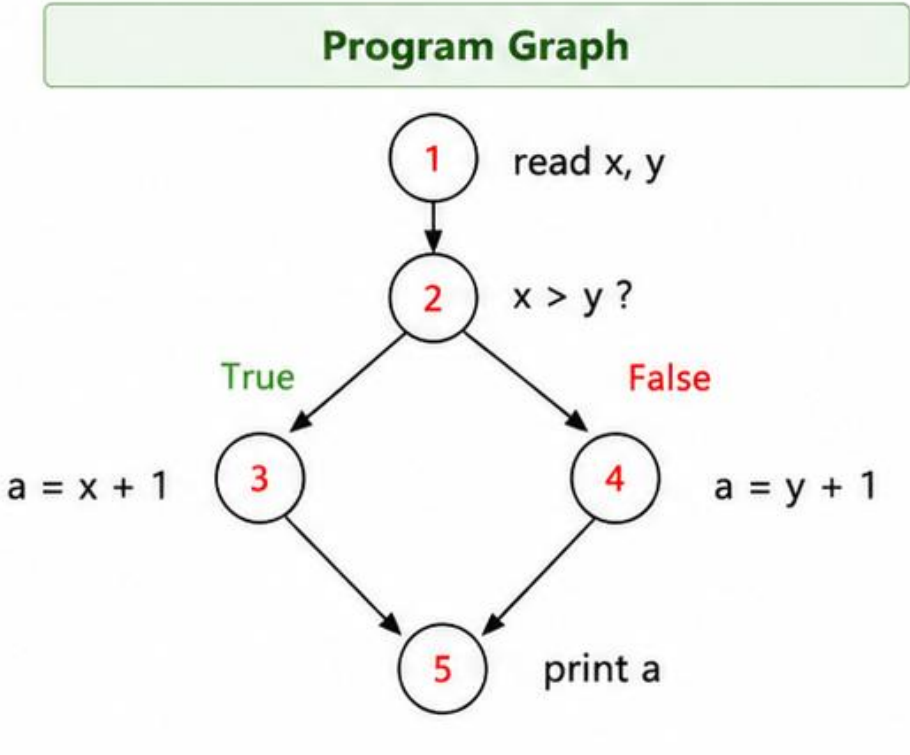
- An example of data flow testing could involve testing a financial application to ensure that the correct data is being processed at each stage of a transaction. For example, **when a customer initiates a transaction, the application must correctly process their account details and the transaction amount.**
- The data must then flow through the different components of the application, such as the authentication module and the payment gateway, without being corrupted or lost..

- To perform data flow testing, testers must identify the different data flows within the application and the different components that process the data.
- They then create test cases that focus on the different flows to ensure that the data is being processed correctly at each stage.

EXAMPLE: DATA FLOW TESTING

Program

1. read x, y;
2. if (x > y)
3. a = x + 1;
4. else
5. a = y + 1;
6. print a;



Define / Use of Variables

VARIABLE	DEFINED AT NODE	USED AT NODE
x	1	2, 3
y	1	2, 4
a	3, 4	5

x and y are defined at node 1 and used in decision at node 2.
a is defined at node 3 or 4 and used at node 5.

UQ24CA342A Software Testing and Automation

Classroom Activity on Data Flow Testing



- 1. read balance
- 2. read amount
- 3. if (amount <= balance)
- 4. balance = balance - amount
- 5. else
- 6. message = "Insufficient Balance"
- 7. print balance/message

Draw the control flow

Variable	Defined at Node	Used at Node
balance		
amount		
message		

Prepare Test cases

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. In Data Flow Testing, what is a definition (def) of a variable?

- A. When a variable is assigned a value
- B. When a variable is deleted
- C. When a variable is printed
- D. When a condition is checked

2. What is a use (use) of a variable?

- A. Assigning a new value to a variable
- B. Using the variable's value in a calculation or decision
- C. Creating a new variable
- D. Removing a variable

3. What is the main focus of Data Flow Testing?

- A. User interface design
- B. Flow of variables through the program
- C. Database performance
- D. Network connectivity

UQ24CA342A Software Testing and Automation

Classroom Quiz

1. A
2. B
3. B

UQ24CA342A Software Testing and Automation

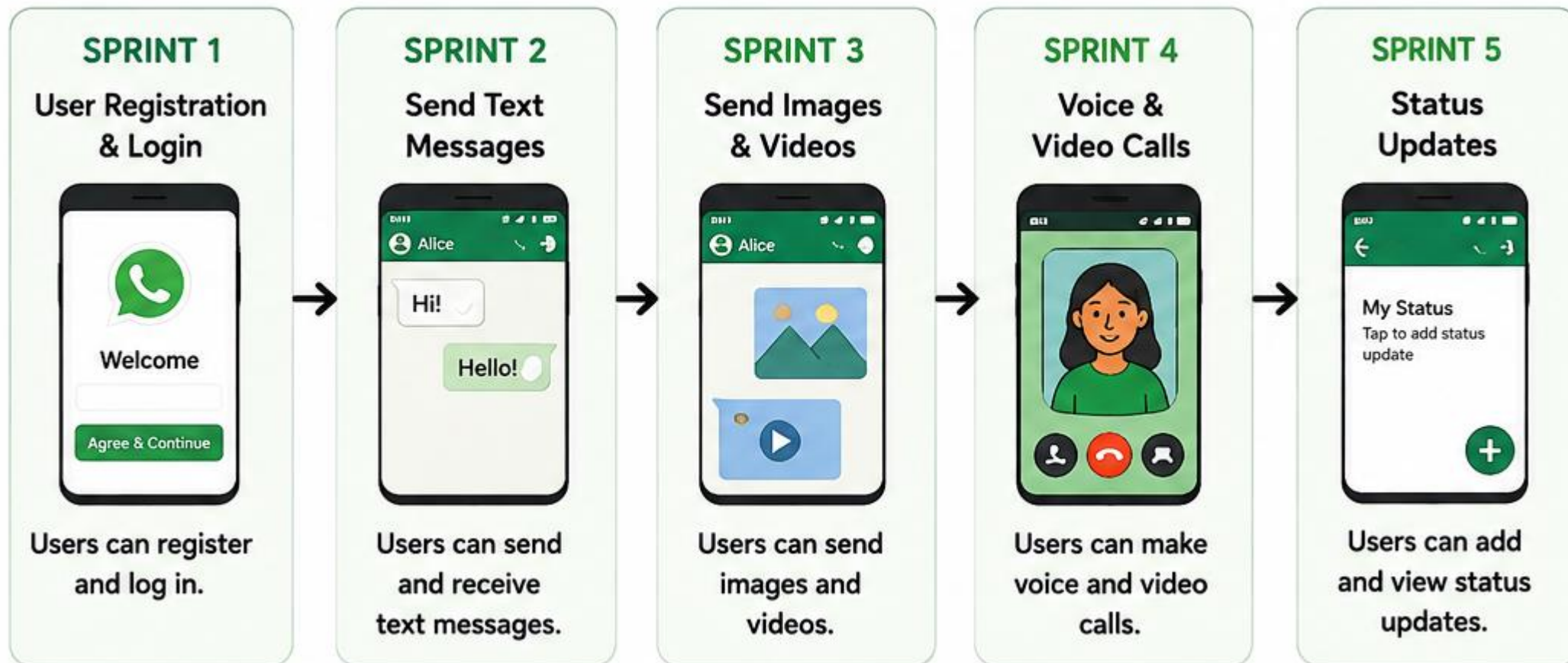
Agile Testing Principles

Mrs. Sowmya BP

Department of Computer Applications

- Agile is a software development approach in which software is developed and delivered in small, frequent increments, while continuously responding to customer feedback and changing requirements.
- Instead of developing the entire software at once, Agile divides the project into smaller parts called iterations or Sprints.

- Agile = Build a little → Test it → Get feedback → Improve it → Build the next part.



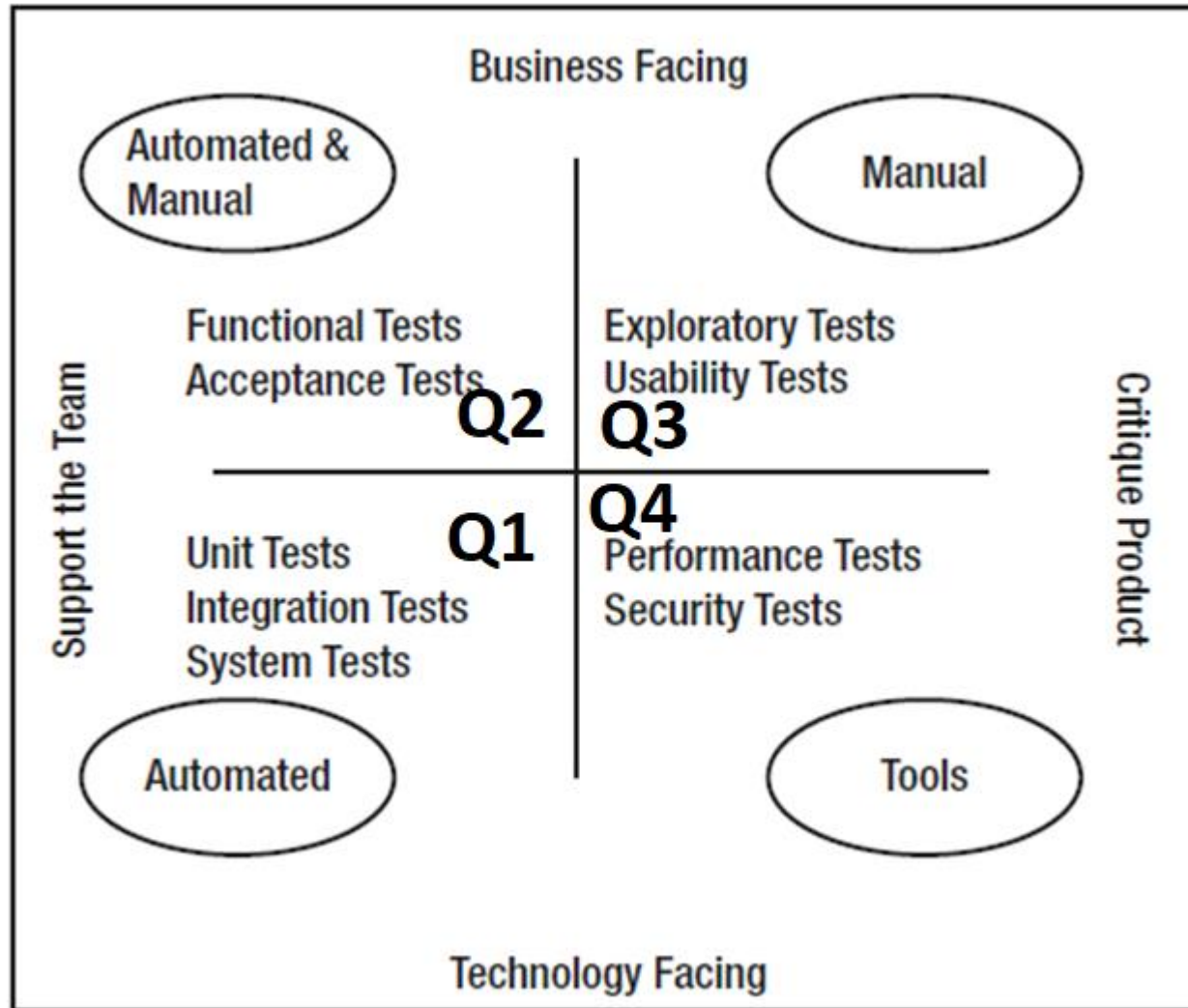
Testing in Agile development is designed to help ensure that the software being developed meets the needs of the customer and is of high quality, while also keeping pace with the fast-paced development cycle of Agile projects.

Some testing principles in Agile development:

- **Collaborate and communicate:** In an Agile environment, collaboration and communication are key. It is important to have a good working relationship with developers, product owners, and other stakeholders to ensure everyone is on the same page.
- **Test early and often:** In Agile development, testing is integrated throughout the development process, not just at the end. This means testing should start as early as possible and should be done frequently.

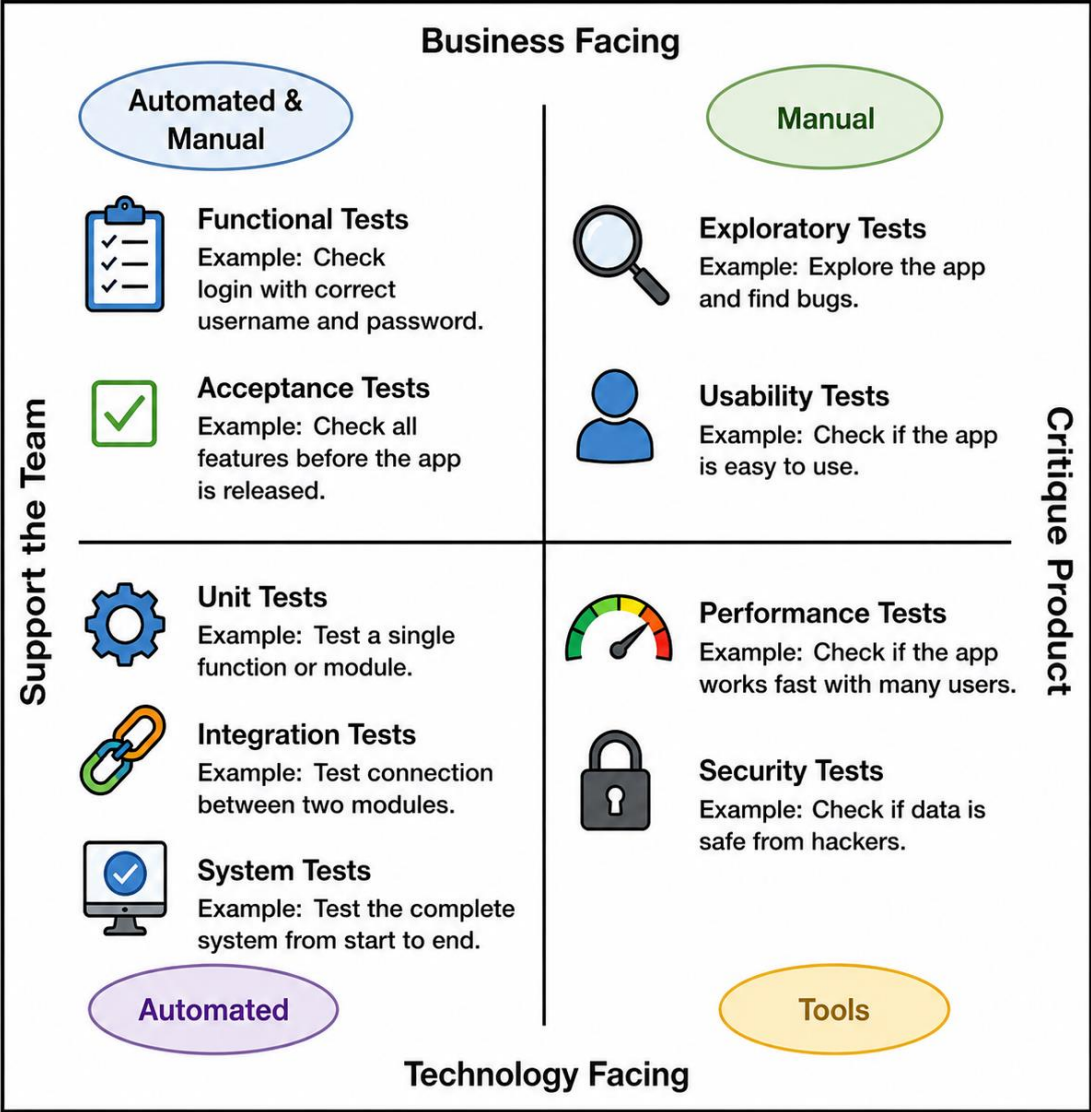
- **Automate testing:** Automated testing is essential in Agile development. It helps ensure that testing is done quickly and efficiently and can be repeated easily as changes are made.
- **Use continuous integration and delivery:** CI/CD is a key part of Agile development. It allows for faster feedback and testing of changes and helps ensure that the software is always in a releasable state.
- **Emphasize exploratory testing:** Exploratory testing is an important part of testing in Agile development. It allows testers to explore the software in a more open-ended and to find issues that may not have been caught by automated or scripted tests.

- **Prioritize testing based on risk:** In an Agile environment, it's important to prioritize testing based on risk. This means focusing on testing the features that are most critical to the business or that have the potential to cause the most harm if they fail.
- **Use metrics to track progress:** Metrics can be used to track the progress of testing in an Agile environment. This can help teams identify areas where they need to improve and can help ensure that testing is on track to meet the goals of the project.
- **Embrace change:** In an Agile environment, change is inevitable. Testers need to be adaptable and able to change their approach as the software evolves. It's important to be flexible and open to new ideas and to be willing to adjust the testing approach as needed.



- **Quadrant Q1:** This quadrant focuses on tests that are technology-driven and are automated. These tests are designed to verify the functionality of the application at a low level. Unit tests, component tests, API tests, and database tests fall under this quadrant.
- **Quadrant Q2:** This quadrant focuses on tests that are business-driven and are automated. These tests are designed to verify the functionality of the application at a high level. Acceptance tests, functional tests, and end-to-end tests fall under this quadrant.

- **Quadrant Q3:** This quadrant focuses on tests that are business-driven and are manual. These tests are designed to verify the application's usability, accessibility, and user experience. Exploratory testing, usability testing, and accessibility testing fall under this quadrant.
- **Quadrant Q4:** This quadrant focuses on tests that are technology-driven and are manual. These tests are designed to verify the nonfunctional aspects of the application such as performance, security, and reliability. Performance testing, security testing, and reliability testing fall under this quadrant.



- Test-driven development (TDD) is a software development approach where test cases are written before the code is developed.
- The idea is to create automated test cases for a small piece of functionality and then write just enough code to make those tests pass.
- Once the code passes the test cases, it is refactored to ensure it is clean and maintainable.

UQ24CA342A Software Testing and Automation

Test-driven development (TDD)



The TDD process involves the following steps:

1. Write a test case: Write an automated test case for a specific functionality or feature.
2. Run the test case: Run the test case and ensure that it fails, since the functionality is not yet implemented.
3. Write the code: Write the code that implements the functionality.
4. Run the test case: Run the test case again and ensure that it passes.
5. Refactor the code: Refactor the code to ensure it is clean and maintainable.

UQ24CA342A Software Testing and Automation

Test-driven development (TDD)



Benefits of TDD are

- **Better quality code:** Since the code is thoroughly tested at each step, it is more reliable and less prone to errors.
- **Faster development:** TDD can help catch errors early, reducing the amount of time spent debugging and fixing issues.
- **Improved collaboration:** TDD can encourage collaboration between developers and testers, as both parties are involved in writing and executing tests.
- **Greater confidence:** TDD can give developers greater confidence in their code, as they know that it has been thoroughly tested and is less likely to fail in production.

UQ24CA342A Software Testing and Automation

Continuous Integration and Continuous Delivery



- CI is a process of regularly integrating code changes into a shared repository, while CD is the process of automatically deploying software changes to production.
- In CI, developers integrate their code changes to a shared repository multiple times a day, rather than waiting for a big merge at the end of the development cycle.
- This ensures that issues are detected and fixed earlier, making the process of code integration and deployment more efficient.
- CI often involves automated build, test, and deployment processes that are triggered by code changes.

UQ24CA342A Software Testing and Automation

Continuous Integration and Continuous Delivery



- CD goes one step further and automates the process of deploying software changes to production. This involves building, testing, and deploying the software changes in a repeatable and reliable manner.
- CD ensures that software is always in a releasable state, making it easier to release new features and bug fixes quickly and frequently.
- Implementing CI/CD requires a combination of tools and practices delivers high-quality software to customers.

UQ24CA342A Software Testing and Automation

Continuous Integration and Continuous Delivery

Benefits of CI/CD:

- **Faster feedback:** CI/CD provides fast feedback on code changes, enabling developers to catch and fix issues early in the development process.
- **Improved collaboration:** CI/CD encourages collaboration between developers, testers, and operations teams, ensuring that everyone is working together to deliver high-quality code.
- **Greater efficiency:** CI/CD automates many of the manual tasks involved in building, testing, and deploying code, freeing up developers to focus on other tasks.
- **Better quality code:** CI/CD ensures that code changes are thoroughly tested and integrated with the existing codebase, resulting in code that is more reliable and less prone to errors.
- **Faster time-to-market:** CI/CD enables organizations to deliver code changes quickly and safely, enabling them to respond more quickly to changing market conditions.

Test automation can help teams to deliver high-quality software quickly and efficiently in Agile development by automating various types of tests, such as

- Unit Testing
- Functional Testing
- Integration Testing
- Acceptance Testing
- Regression Testing



THANK YOU

Mrs. Sowmya BP

Department of Computer Applications

sowmyabp@pes.edu